

Содержание

Введение	5
1. Анализ технического задания	6
1.1. Описание предметной области	10
1.2. Аналогии	12
2. Модели данных	14
2.1. Концептуальная модель	15
2.2. Логическая модель	20
2.3. Приведение логической модели к физической	23
3. Разработка и реализация АИС	32
3.1. Создание SQL-запросов	32
3.2. Руководство пользователя	40
3.3. Руководство программиста	51
4. Тестирование АИС	59
Заключение	Ошибка! Закладка не определена.
Список литературы	66
Приложение А: «Модели данных»	67
Приложение Б: Текст кода	70

					МИВУ 10.03.01-12ПЗ			
Изм.	Лист	№ докум	Подпись	Дата				
Разраб		Рысев А.С.			АИС отдела кредитования	Литера	Лист	Листов
Пров		Колпаков А. А.				у	4	
						МИВлГУ ИБ-122		
Н. Контр.								
Утв		Орлов А. А.						

Введение

В современном банковском секторе кредитование является одним из ключевых направлений деятельности финансовых учреждений, обеспечивающим устойчивое развитие экономики. Широкий спектр кредитных продуктов, необходимость анализа данных о клиентах, оценка их платежеспособности и управление рисками создают сложные задачи для отдела кредитования. Ручная обработка большого объема информации замедляет принятие решений, снижает оперативность работы и повышает вероятность ошибок, что негативно сказывается на эффективности кредитных процессов. В условиях растущей конкуренции и повышения требований к оперативности и надежности работы финансовых организаций внедрение автоматизированных систем управления становится необходимым. Использование информационных систем для работы с базами данных позволяет эффективно хранить и обрабатывать информацию о заемщиках, кредитных продуктах, операциях и отчетах. Это дает возможность быстро получать данные, необходимые для анализа, минимизировать риски и улучшить управление кредитным портфелем. Целью данной курсовой работы является разработка информационной системы для автоматизации работы отдела кредитования, которая будет обеспечивать удобное взаимодействие сотрудников с базой данных и повышать эффективность обработки информации.

Для достижения данной цели поставлены следующие задачи:

1. Исследовать существующие СУБД и оценить их производительность в контексте задач кредитования.
2. Разработать концептуальную и логическую модели данных, отражающие структуру кредитных процессов.
3. Спроектировать базу данных для учета клиентов, кредитных договоров и финансовых операций.
4. Создать многопользовательское приложение для работы с базой данных, обеспечивающее удобный интерфейс и функции для обработки запросов

1. Анализ технического задания

Современная разработка информационных систем требует тщательного выбора инструментов, наиболее подходящих для реализации требований проекта. При создании системы управления базой данных для отдела кредитования важно рассмотреть и сравнить такие технологии, как языки программирования C# и C++, среды разработки Visual Studio и Eclipse, а также СУБД SQLite и MySQL. Данный анализ позволит определить оптимальные инструменты, основываясь на их производительности, удобстве использования, гибкости, безопасности и возможности интеграции с другими системами. Это обеспечит надежность и эффективность создаваемой системы, соответствующей высоким стандартам разработки и эксплуатации.

1. Характеристики C++ [1]:

- Производительность: Высокая, благодаря низкоуровневым возможностям языка, включая управление памятью и системными ресурсами.
- Гибкость: Предоставляет детализированный контроль над аппаратным обеспечением.
- Применение: Используется для создания системного ПО, драйверов и высокопроизводительных приложений.

2. Характеристики C# [2]:

- Простота: Язык высокого уровня, проще для освоения и использования по сравнению с C++.
- Платформа: Тесно интегрирован с .NET, что упрощает разработку веб-приложений и бизнес-логики.
- Безопасность: Обладает автоматическим управлением памятью (сборка мусора), снижая вероятность ошибок, связанных с утечкой памяти.

3. Характеристики Visual Studio [5]:

- **Функциональность:** Включает обширный набор инструментов для отладки, тестирования и разработки.
- **Интеграция:** Отлично взаимодействует с экосистемой Microsoft, поддерживает множество языков программирования.
- **Удобство:** Предоставляет интуитивно понятный интерфейс.

4. Характеристики Eclipse [6]:

- **Открытый исходный код:** Бесплатная, поддерживаемая сообществом среда разработки.
- **Кроссплатформенность:** Поддерживает работу на различных ОС (Windows, macOS, Linux).
- **Плагины:** Большое количество доступных плагинов для расширения возможностей.

5. Характеристики SQLite [3]:

- **Легковесность:** Не требует установки и настройки сервера.
- **Простота:** Имеет компактную и легкую в использовании архитектуру.
- **Встроенность:** Оптимальна для приложений, где необходимо локальное хранилище данных.

6. Характеристики MySQL [4]:

- **Производительность:** Высокая скорость работы и масштабируемость для крупных проектов.
- **Функциональность:** Поддерживает сложные запросы, транзакции и интеграцию с другими СУБД.
- **Популярность:** Широко используется в веб-разработке, поддерживается большим сообществом.

Результат анализа характеристик можно представить в виде таблиц: 1 – таблица сравнения языков программирования, 2 – таблица сравнения сред, 3 – таблица сравнения СУБД.

Таблица 1 – Сравнение языков программирования

Критерий	C++	C#
Производительность	Высокая	Средняя
Простота	Сложная	Простая
Гибкость	Высокая	Средняя
Интеграция	Ограниченная	Отличная с .NET
Безопасность	Низкая (ручное управление памятью)	Высокая (автоматическая сборка мусора)

Таблица 2 – Сравнение сред программирования

Критерий	Visual Studio	Eclipse
Производительность	Высокая	Высокая
Простота	Удобная	Средняя
Гибкость	Высокая	Высокая
Интеграция	Отличная с экосистемой MS	Кроссплатформенная
Безопасность	Высокая	Высокая

Таблица 3 – Сравнение СУБД

Критерий	SQLite	MySQL
Производительность	Высокая	Высокая
Простота	Простая	Средняя
Гибкость	Низкая	Высокая
Интеграция	Локальное хранилище данных	Веб-разработка
Безопасность	Высокая	Высокая

В данном конкретном случае C#, Visual Studio и SQLite являются оптимальными выбором. Они предлагают следующие преимущества:

- C#: Простота использования, высокая производительность, отличная совместимость с .NET и высокая безопасность. [2]
- Visual Studio: Мощная и интуитивно понятная интегрированная среда разработки, поддержка множества языков и богатый набор инструментов. [5]
- SQLite: Легковесная и простая в использовании СУБД, идеально подходящая для встроенных систем и приложений с локальным хранением данных. [3]

Эти инструменты обеспечат стабильную, безопасную и эффективную работу системы управления базой данных для отдела кредитования.

1.1. Описание предметной области

Предметная область — часть реального мира, данные о которой мы хотим отразить в базе данных. В данной курсовой работе рассматривается предметная область «Отдел кредитования». Основной задачей является проектирование и разработка приложения для автоматизации процесса оформления кредитов. Предполагаемая база данных должна обеспечивать работу отдела кредитования по учету выданных кредитов различного типа, автоматизированную генерацию отчетов по операциям за определённые периоды и по категориям кредитов.

Процесс оформления кредита осуществляется следующим образом:

1. Поиск клиентов. Встреча с потенциальными заемщиками для выяснения их потребностей и финансовых возможностей. Эту задачу выполняют кредитные консультанты.

На данном этапе клиент знакомится с предложениями отдела кредитования в зависимости от своих запросов и финансовой ситуации.

2. Оформление кредита. При заключении договора главный менеджер или кредитный менеджер регистрируют клиента в базе.

Для оформления кредита клиенту необходимо предоставить номер телефона, а также свои паспортные данные.

3. Заключение сделки. Оформление договора может производить кредитный менеджер или главный менеджер.

В отделе кредитования работают различные сотрудники: кредитные консультанты, кредитные менеджеры и главный менеджер. Главный менеджер контролирует работу отдела.

Клиент, обращаясь в отдел кредитования, взаимодействует с консультантом, который помогает выбрать наиболее подходящий кредитный продукт и оформляет сделку. При выдаче кредита клиент получает договор с указанием

условий кредита (сумма, процентная ставка, срок), данных о менеджере, заключившем сделку, и графиком платежей.

Исходя из данных требований к приложению, база данных должна содержать несколько таблиц, а данные таблиц должны быть связаны для обеспечения целостности системы. Для разработки программы необходимо учитывать, что она создается, прежде всего, для пользователя. Поэтому ключевым требованием является удобный и интуитивно понятный интерфейс. Нужно предусмотреть все возможности управления приложением, чтобы максимально упростить работу пользователя и повысить эффективность системы.

Программа должна корректно работать с данными, обеспечивая вывод точного результата по запросу пользователя. Она должна мгновенно реагировать на действия пользователя и предоставлять требуемую информацию в зависимости от его запроса.

1.2. Аналоги

В настоящее время реализовано огромное количество АС для отделов кредитования. В настоящий момент существует CrediMax Pro – система управления процессами кредитования, предназначенная для автоматизации работы кредитных организаций.

Достоинства:

- Удобный интерфейс для обработки заявок и управления кредитными портфелями.
- Поддержка различных видов кредитов (потребительские, ипотечные, автокредиты и т. д.).
- Интеграция с бюро кредитных историй для проверки заемщиков.
- Автоматическое формирование графиков платежей и напоминаний для клиентов.
- Гибкая настройка бизнес-процессов под конкретные требования организации.

Недостатки:

- Высокая стоимость внедрения и технической поддержки.
- Необходимость обучения персонала для работы с системой.
- Ограниченные возможности кастомизации для уникальных процессов кредитной организации.

Особенности:

- Модули управления заявками, кредитными портфелями и рисками.
- Поддержка аналитики для прогнозирования возвратов и дефолтов.
- Возможность интеграции с внешними банковскими системами и платежными шлюзами.
- Веб-интерфейс и мобильное приложение для заемщиков.

					МИВУ 10.03.01-12ПЗ	Лист
Изм.	Лист	№ докум	Подпись	Дата		12

В качестве другого примера можно привести LoanHub CRM – платформа для управления клиентами и процессами кредитования в финансовых учреждениях.

Достоинства:

- Централизованное хранение данных о клиентах и их кредитах.
- Интеграция с банковскими системами и платежными платформами.
- Инструменты анализа и отчетности по клиентам и финансовым операциям.
- Возможность настройки автоматизированных уведомлений и обработки заявок.
- Облачное решение, не требующее сложной инфраструктуры.

Недостатки:

- Абонентская плата за использование облачного сервиса.
- Ограниченная гибкость для крупных организаций с уникальными процессами.
- Зависимость от интернет-соединения для работы.

Особенности:

- Модули управления клиентами, заявками на кредит и просроченными задолженностями.
- Встроенные инструменты для оценки рисков и автоматического скоринга.
- Поддержка интеграции с мобильными приложениями и веб-платформами.
- Панели мониторинга для анализа эффективности работы отдела.

2. Модели данных

Данный этап является самым важным при создании АИС. Здесь выделяются сущности, атрибуты сущностей и связи между сущностями. На основе полученной диаграммы “Сущность – связь” или логической модели строятся функциональные модели системы и диаграмма потоков данных. Для создания базы данных, нужно логическую модель представить в виде физической. [7]

Разработка моделей данных включает 3 этапа: [7]

1) Концептуальная модель:

- определение сущностей (объектов, понятий) предметной области и их взаимосвязей;
- выявление атрибутов сущностей и описание их характеристик;
- построение диаграммы сущность-связь (ER-диаграммы) для визуального представления модели.

2) Логическая модель:

- трансформация концептуальной модели в структуру, подходящую для конкретной СУБД;
- определение типов данных, ключей, индексов, ограничений целостности;
- нормализация данных для устранения избыточности и аномалий;
- спецификация отношений между сущностями (таблицами).

3) Физическая модель:

- оптимизация логической модели для конкретной СУБД и аппаратной платформы;
- определение физического хранения данных: файлов, томов, буферов и т.д.;
- результат - физическая схема базы данных, готовая для реализации.

2.1. Концептуальная модель

Концептуальная модель данных является первым и наиболее абстрактным уровнем проектирования базы данных. На этом этапе определяются ключевые сущности предметной области, их атрибуты и взаимосвязи. [7]

В ходе анализа процесса оформления кредита, были выделены следующие базовые сущности организации и их атрибуты:

1) Сотрудник организации.

Атрибуты персонала: ID, ФИО, Должность, Телефон, Стаж. Они представлены на рисунке 1.

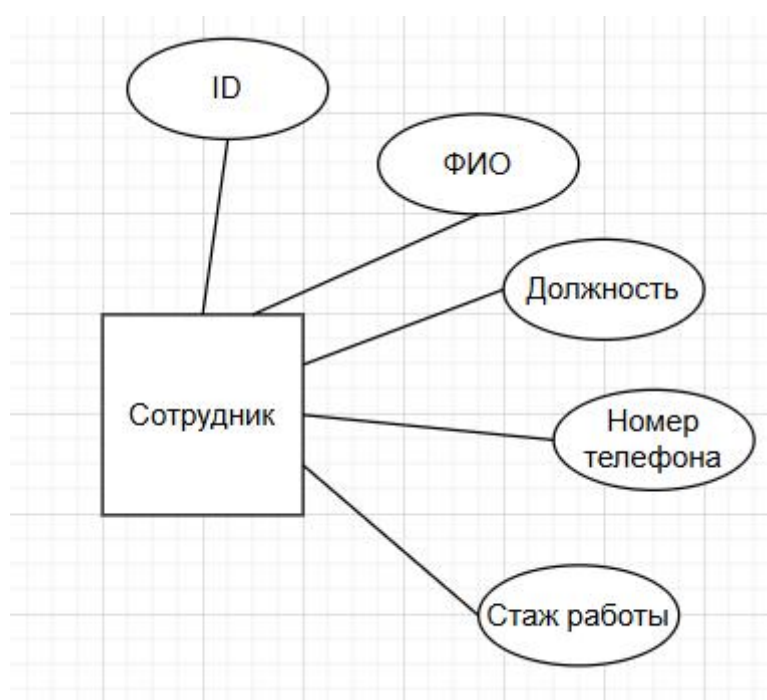


Рис. 1 – Атрибуты сотрудника

2) Кредит.

Атрибуты кредита: ID, ID_Платежа, Банк, Цель кредита, Срок кредита, Статус кредита, Дата выдачи кредита, Сумма кредита, Процентная ставка. Доступность кредита представлены на рисунке 2.

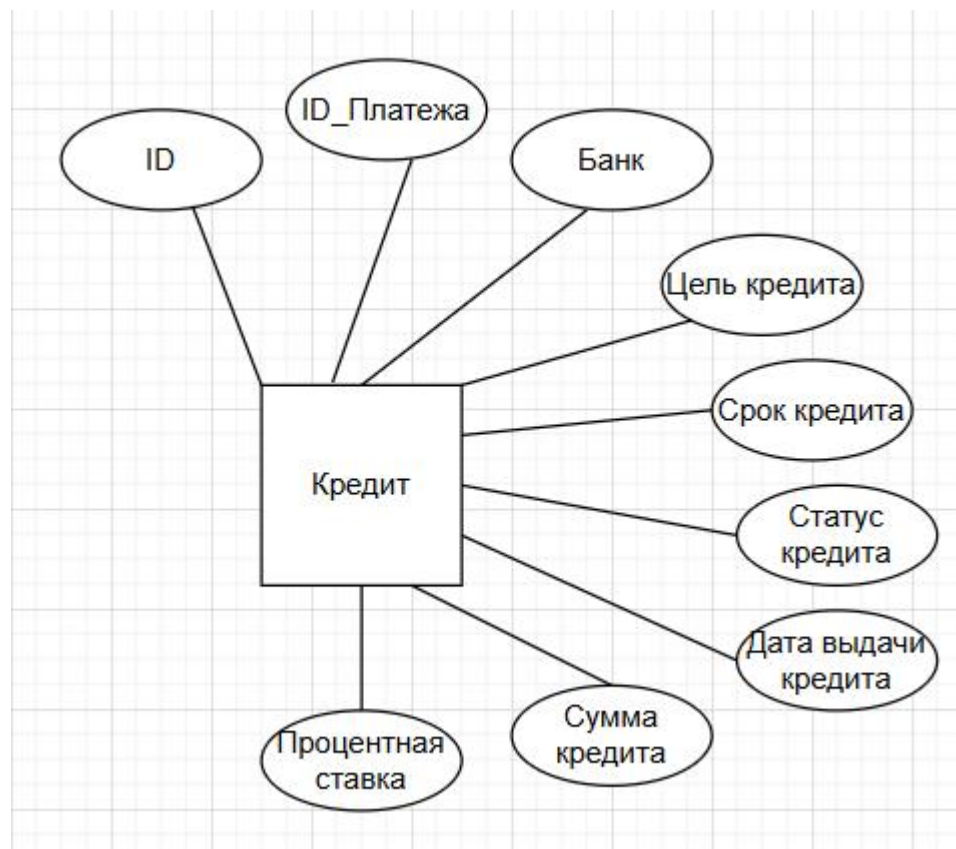


Рис. 2 – Атрибуты кредита

3) Договор.

Атрибуты договора: ID, Фамилия, Имя, Отчество, Дата рождения, Серия паспорта, Номер паспорта, ИНН, Адрес проживания, Номер телефона, Банк, Сумма кредита, Срок кредита, Процентная ставка, Цель кредита, ID_Кредита, ID_Клиента, ID_Сотрудника.

Время окончания аренды можно увидеть на рисунке 3.

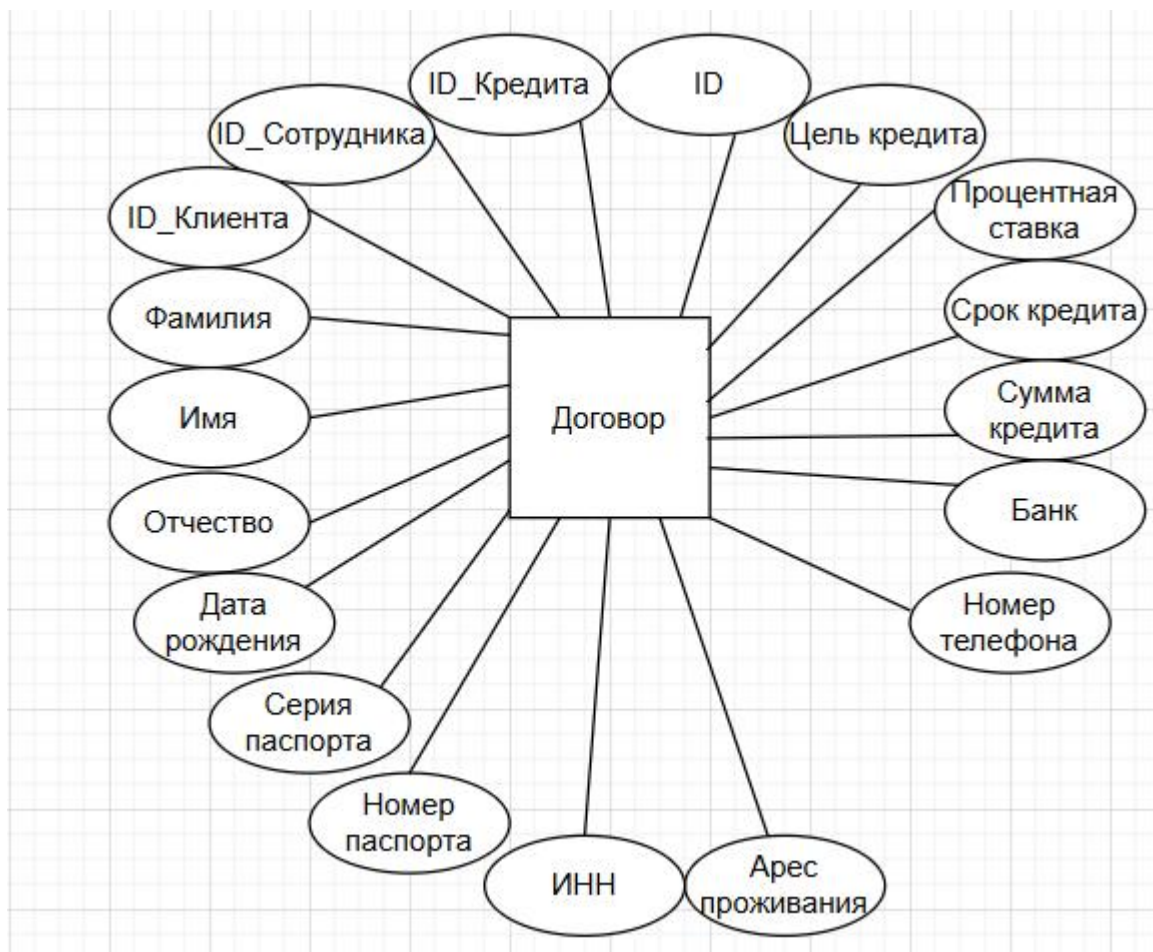


Рис. 3 – Атрибуты договора

4) Клиенты.

Атрибуты клиента: ID, Номер паспорта, Серия паспорта, Фамилия, Имя, Отчество, Номер телефона, ИНН, Адрес проживания, Дата рождения, Дата регистрации изображены на рисунке 4.

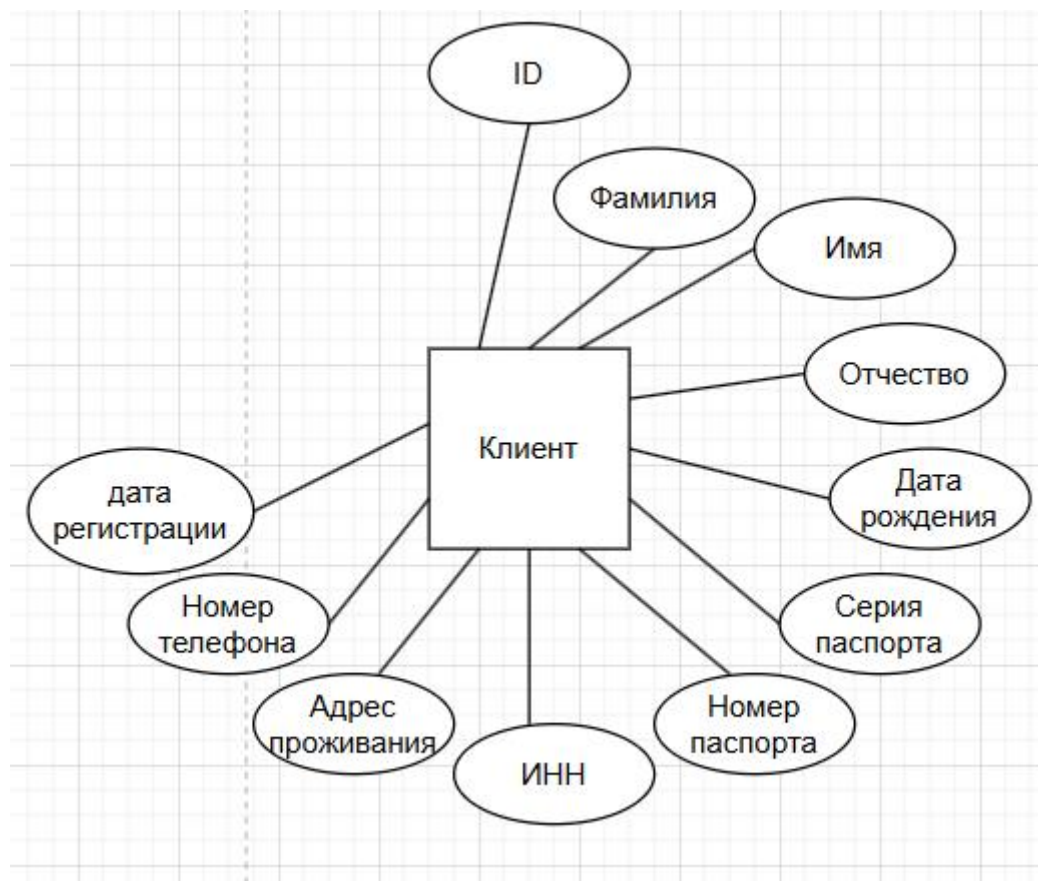


Рис. 4 – Атрибуты клиента

5) Платеж.

Атрибуты платежа: ID, ФИО, Дата платежа, Вид платежа, Сумма платежа, Сумма кредита, Остаток долга, Статус кредита изображены на рисунке 5.

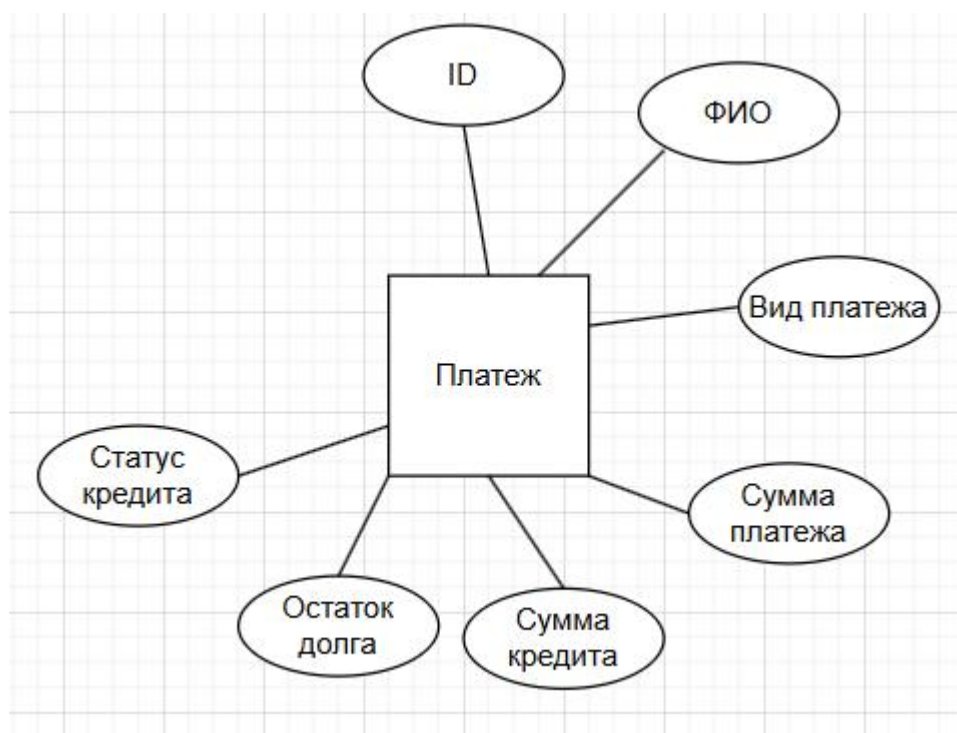


Рис. 5 – Атрибуты платежа

Все выделенные сущности были объединены в общую концептуальную модель данных, которая представлена на рисунке 5.

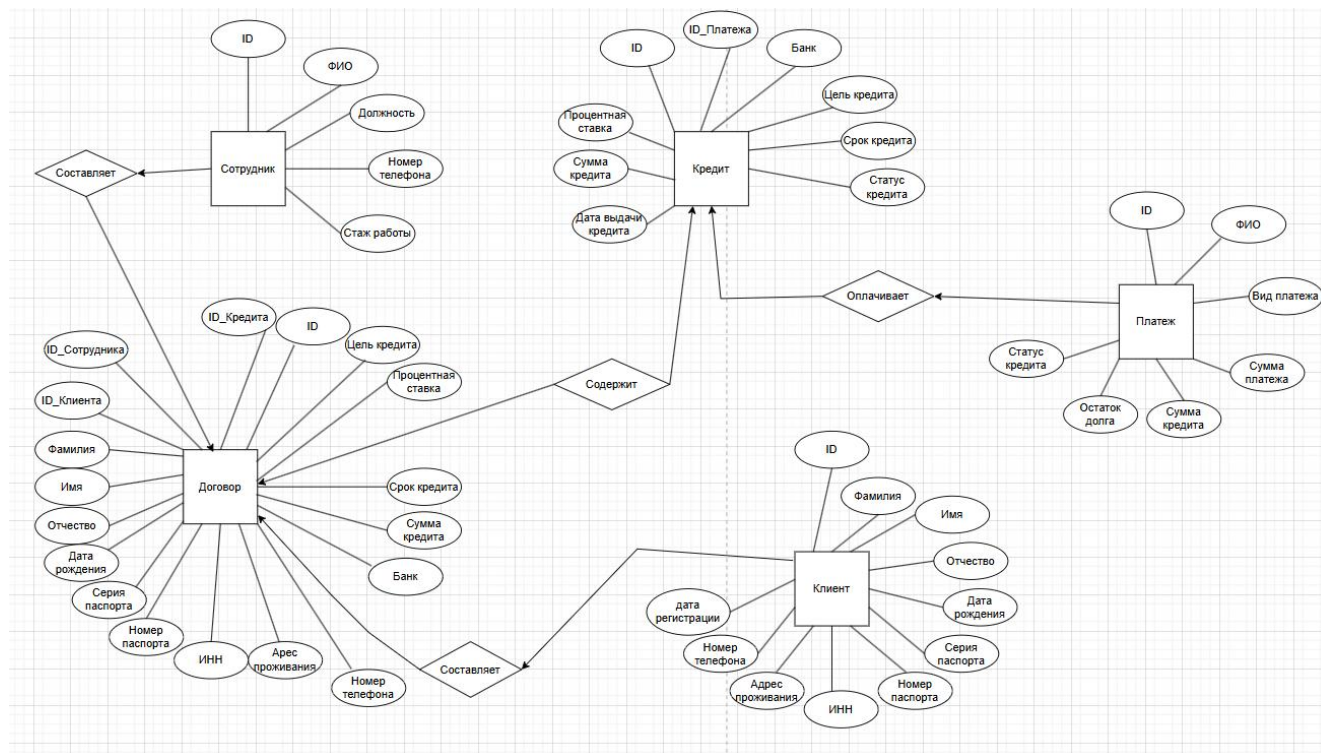


Рис. 6 – Концептуальная модель данных.

Можно выделить следующие связи между сущностями:

Между всеми таблицами существуют связи один ко многим.

2.2. Логическая модель

Логическая модель данных — это абстрактное представление структуры данных, которое используется для планирования и проектирования баз данных. [7]

Ее создание начинается с анализа предметной области и выделения сущностей.

Данная логическая модель данных отражает основные сущности, связанные с Отделом кредитования: клиенты, договор, кредит, платеж и сотрудники. Она позволяет хранить и управлять данными, необходимыми для эффективного функционирования отдела кредитования

Основные сущности и их атрибуты:

Клиенты:

- ID – Уникальный идентификатор клиента, первичный ключ
- Фамилия
- Имя
- Отчество
- Серия_паспорта
- Номер_паспорта
- Дата_рождения
- ИНН
- Номер_телефон
- Дата_регистрации

Договор:

- ID_Кредита – внешний ключ
- ID_Клиента – внешний ключ
- ID_Сотрудника – внешний ключ
- ID
- Фамилия
- Имя
- Отчество
- Дата_рождения
- Серия_паспорта

- Номер_паспорта
- ИНН
- Адрес_проживания
- Номер_телефона
- Банк
- Сумма_кредита
- Срок_кредита
- Процентная_ставка
- Цель_кредита

Кредит:

- ID – уникальный идентификатор, первичный ключ
- ID_Платежа – уникальный идентификатор платежа, внешний ключ
- Банк
- Цель_кредита
- Срок_кредита
- Статус_кредита
- Дата_выдачи_кредита
- Сумма_кредита
- Процентная_ставка

Платеж:

- ID – уникальный идентификатор, первичный ключ
- Фамилия
- Имя
- Отчество
- Дата_платежа
- Вид_платежа
- Сумма_кредита
- Сумма_платежа
- Остаток_долга
- Статус_кредита

					МИВУ 10.03.01-12ПЗ	Лист
						21
Изм.	Лист	№ докум	Подпись	Дата		

На основе этих данных была составлена логическая модель данных, представленная на рисунке 7.

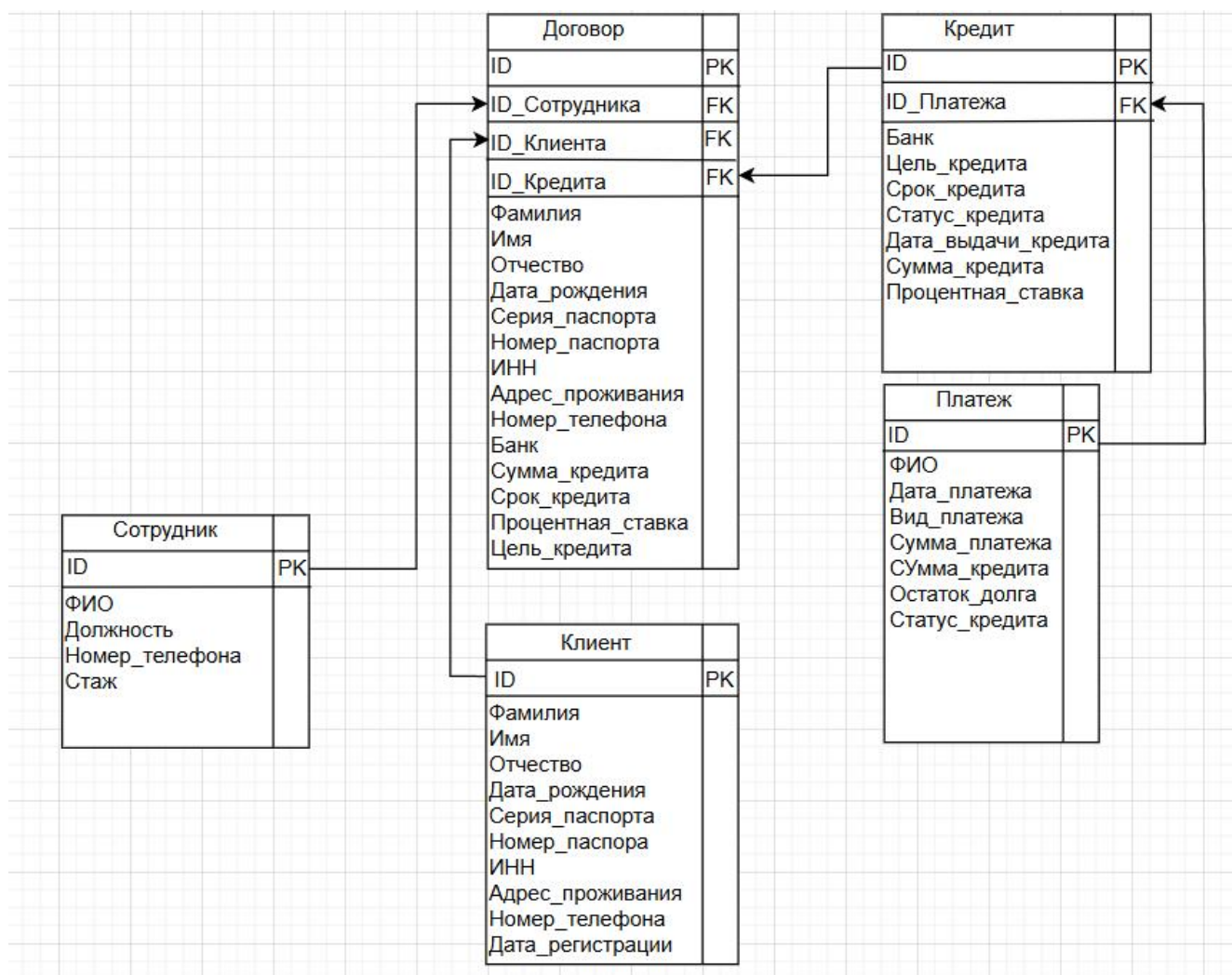


Рис. 7 – Логическая модель данных

2.3. Приведение логической модели к физической

Физическая модель данных — это заключительный шаг в процессе моделирования данных. Она представляет фактические детали реализации в конкретной СУБД. [7]

К основным задачам физической модели данных относятся:

1) Определение физических характеристик таблиц и полей:

- Типы данных для каждого атрибута (INTEGER, TEXT, DATE и т.д.)
- Размеры полей (длина строк)
- Ограничения целостности данных (первичные/внешние ключи, уникальность, проверка значений)

2) Проектирование физической структуры базы данных:

- Определение физического хранения данных (файлы, тома, блоки)
- Настройка индексов для ускорения доступа к данным
- Решения по кластеризации таблиц
- Механизмы резервного копирования и восстановления данных

3) Оптимизация производительности:

- Анализ запросов и создание оптимальных индексов
- Денормализация данных для повышения скорости выборки
- Использование материализованных представлений

Для создания этой модели были созданы таблицы и их столбцы:

Клиент:

ID(INTEGER), Имя(TEXT(20)), Фамилия(TEXT(20)), Отчество(TEXT(20)),
Дата_рождения(TEXT(20)), Серия_паспорта(NUMERIC(4)),
Номер_паспорта(NUMERIC(6)), ИНН(NUMERIC(8)),
Адрес_проживания(TEXT(20)), Номер_телефона(NUMERIC(11)),
Дата_регистрации(DATE)

Аналогично этому были созданы и другие таблицы и столбцы

Физическую модель можно преобразовывать по частям. Поле «ID» для таблицы «Клиент» связано с полем «ID_Клиента» в таблице «Договор». Данные

поля также являются и первичным ключом для таблицы «Клиент» и внешним ключом для таблицы «Договор». Связь между двумя таблицами представлена на рисунке 8.

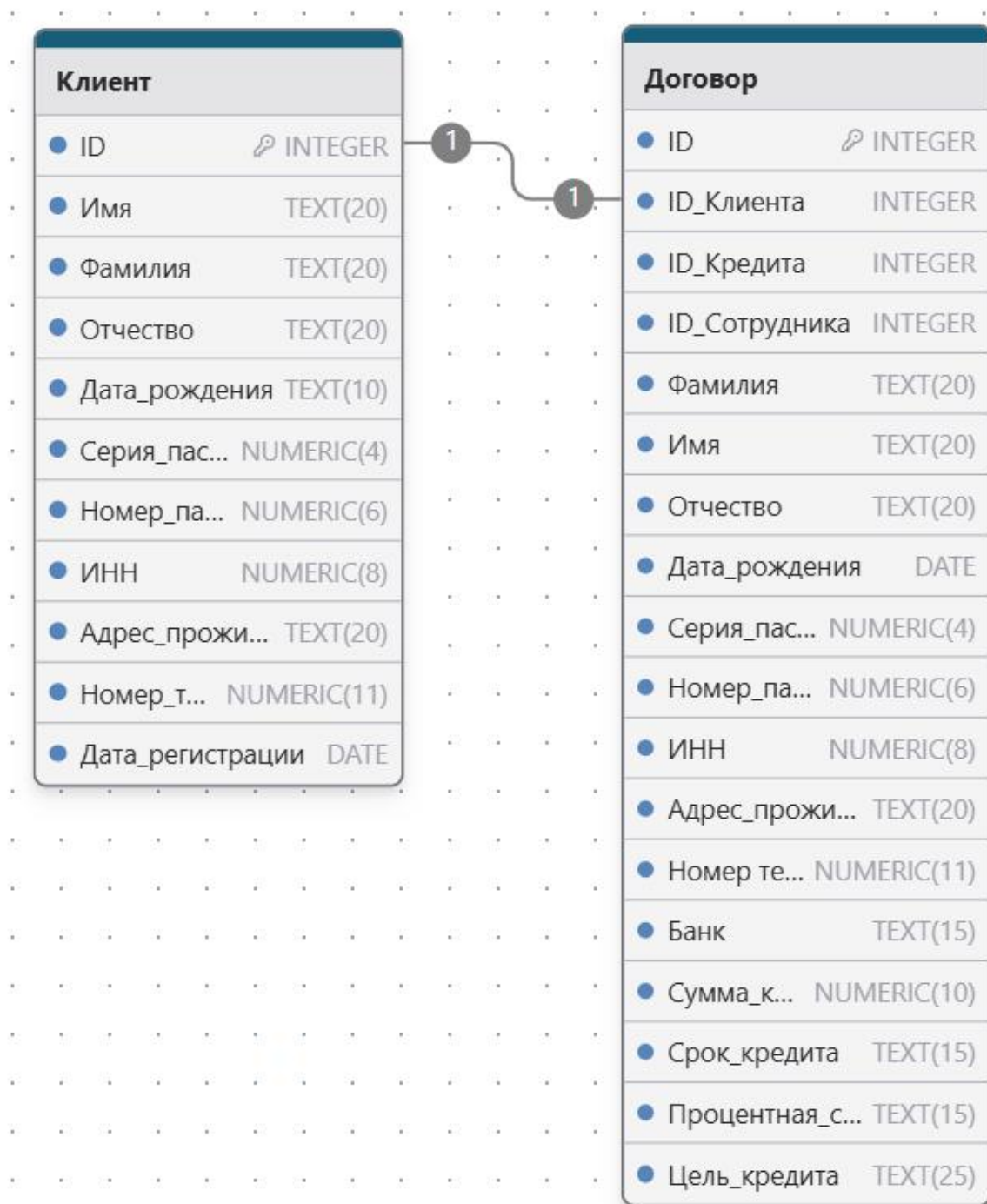


Рис. 8 – Связь между таблицами «Клиент» и «Договор»

В таблице «Сотрудник» ключевым также будет поле «ID», оно будет иметь связь с полем «ID_Сотрудника» в таблице «Договор». По аналогии с таблицей «Клиенты» эти поля будут первичным и внешним ключами. Связь показана на рисунке 9.

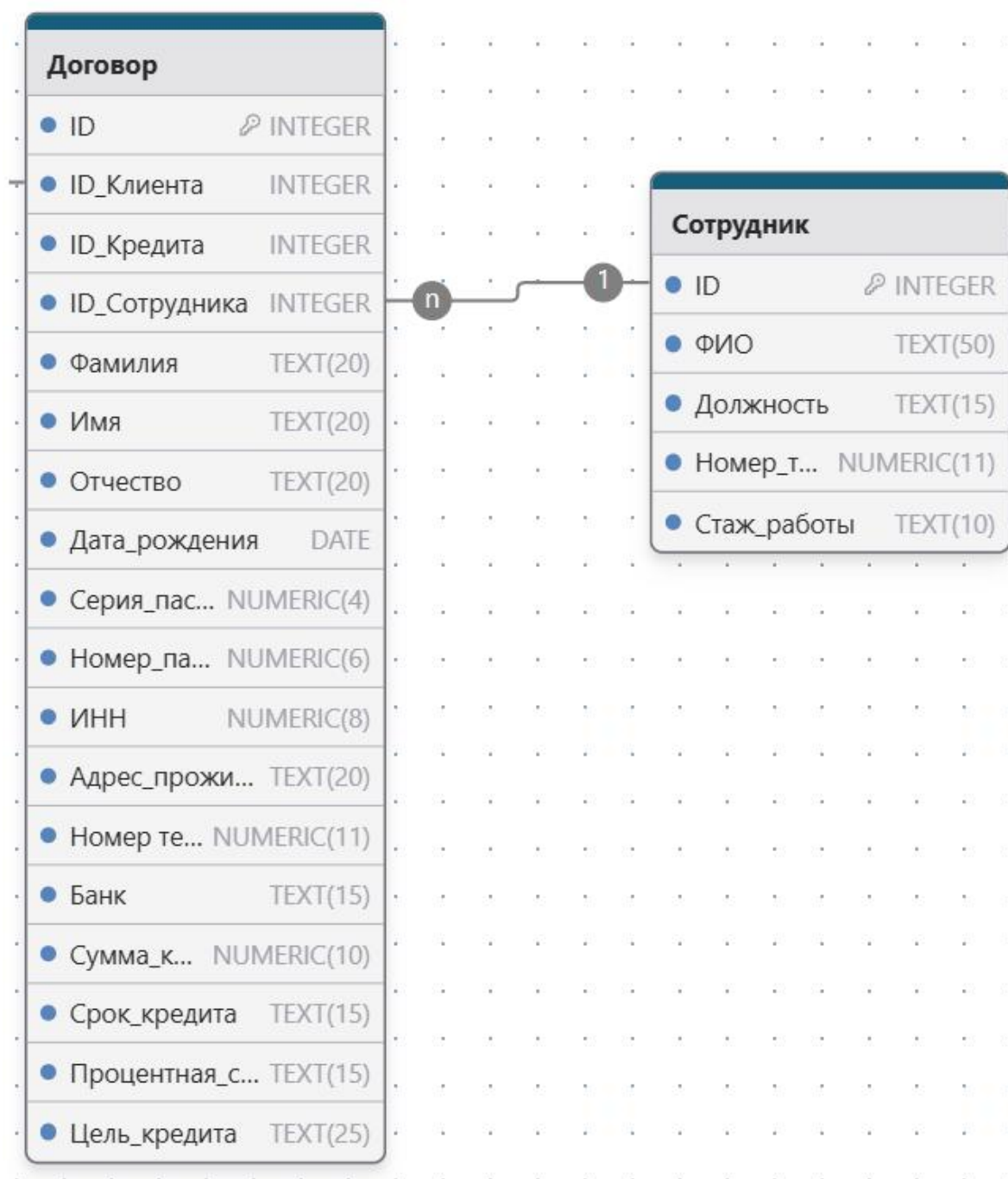


Рис. 9 – Связь между таблицами «Сотрудник» и «Договор»

Связь между таблицами «Кредит» и «Договор» организована через поля «ID» и «ID_Кредита» соответственно. Это можно увидеть на рисунке 10.

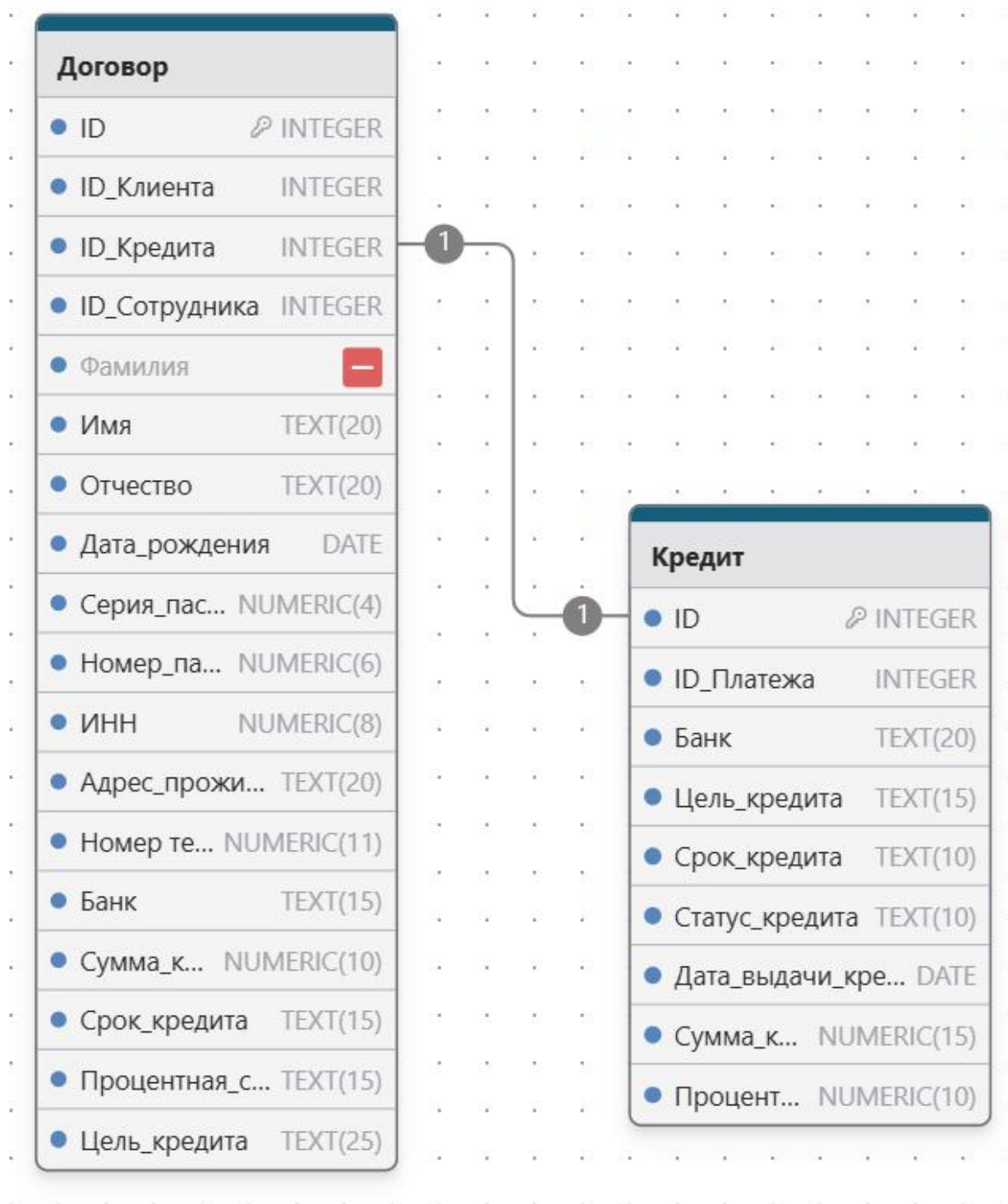


Рис. 10 – Связь между таблицами «Кредит» и «Договор»

Также таблица «Кредит» связана с еще одной таблицей: «Платеж», связана она через поле ID в таблице «Платеж», и ID_Платежа в таблице «Кредит». Это можно увидеть на рисунке 11.

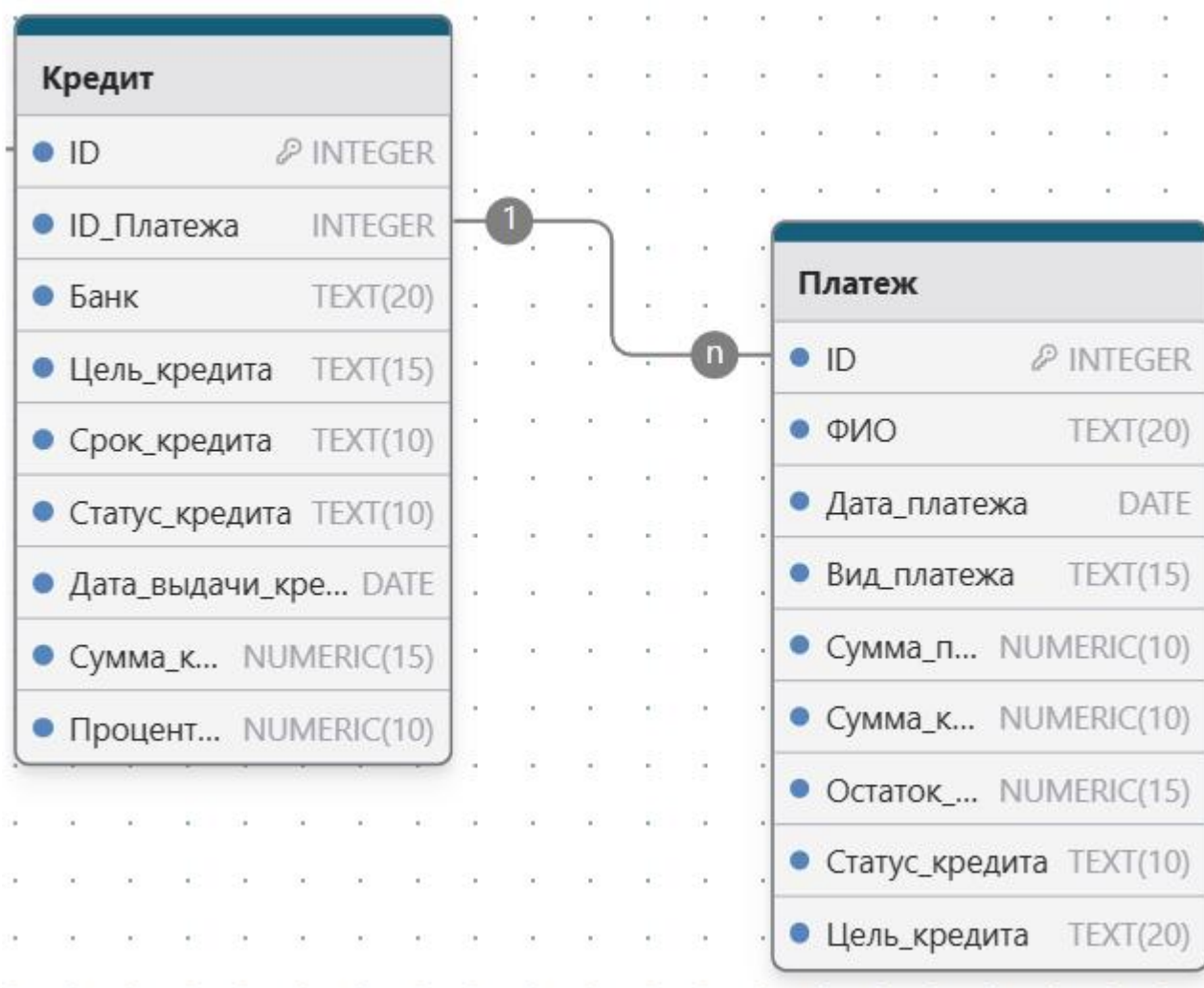


Рис. 11 – Связь между таблицами «Кредит» и «Платеж»

Полная схема физической модели данных представлена на рисунке 12.

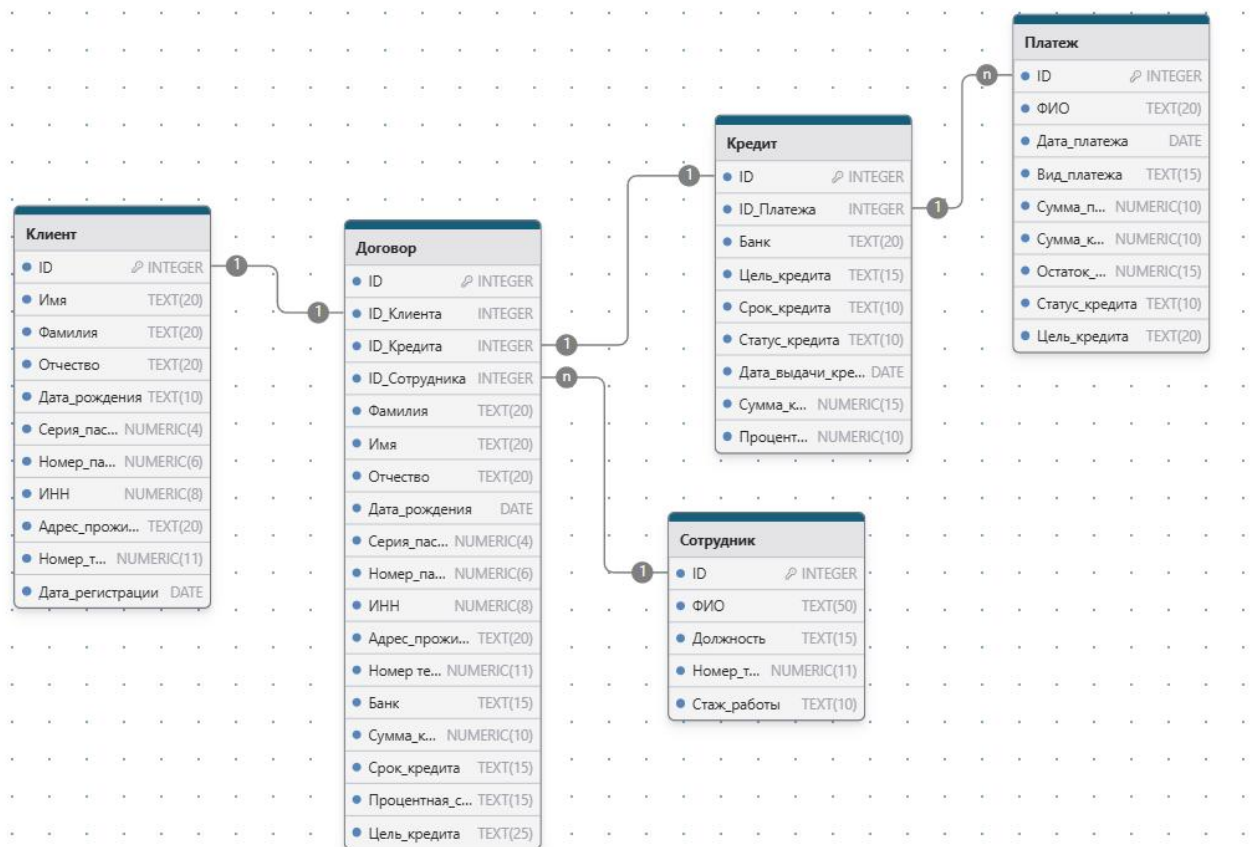


Рис. 12 – Физическая модель данных

Таблица "Клиент":

ID_PK (INTEGER) – Уникальный идентификатор, первичный ключ, автоувеличение

Имя(TEXT(20))

Фамилия(TEXT(20))

Отчество(TEXT(20))

Дата_рождения(TEXT(10))

Серия_паспорта(NUMERIC(4))

Номер_паспорта(NUMERIC(6))

ИНН(NUMERIC(8))

Адрес_проживания(TEXT(20))

Номер_телефона(NUMERIC(11))

Дата_регистрации(DATE)

Для данной таблицы могут быть определены:

- первичный ключ ID для уникальной идентификации записей;
- индекс по полю ИНН для ускорения поиска клиентов по ИНН;
- ограничение уникальности для Серия, Номер паспорта и Номер_телефона

Таблица "Договор":

ID_Клиента_FK (INTEGER), внешний ключ на таблицу Клиент)

ID_Сотрудника_FK (INTEGER), внешний ключ на таблицу Сотрудник)

ID_Кредита_FK (INTEGER), внешний ключ на таблицу Кредит)

ID (INTEGER) - Уникальный идентификатор, первичный ключ,

автоувеличение

Фамилия(TEXT(20))

Имя(TEXT(20))

Отчество(TEXT(20))

Дата_рождения(DATE)

Серия_паспорта(NUMERIC(4))

Номер_паспорта(NUMERIC(6))

ИНН(NUMERIC(8))

Адрес_проживания(TEXT(20))

Номер_телефона(NUMERIC(11))

Банк(TEXT(15))

Сумма_кредита(NUMERIC(10))

Срок_кредита(TEXT(15))

Процентная_ставка(TEXT(15))

Цель_кредита(TEXT(25))

Ключевые особенности:

- поле ID для уникальной идентификации записей
- связь с таблицами Клиенты, Сотрудник и Кредит через внешние ключи;
- индекс по ID_Клиент_FK для ускорения поиска клиента, оформившего кредит;

Таблица "Сотрудник":

ID_PK (INTEGER) – уникальный идентификатор, первичный ключ, автоувеличение

ФИО(TEXT(50))

Должность(TEXT(15))

Номер_телефона(NUMERIC(11))

Стаж_работы(TEXT(10))

Ключевые особенности:

-первичный ключ ID для уникальной идентификации записей;

Таблица «Кредит»

ID_PK (INTEGER) – Уникальный идентификатор, первичный ключ

ID_Платежа_FK (INTEGER) - внешний ключ на таблицу Платеж

БАНК(TEXT(20))

Цель_кредита(TEXT(15))

Срок_кредита(TEXT(10))

Статус_кредита(TEXT(10))

Дата_выдачи_кредита(DATE)

Сумма_кредита(NUMERIC(15))

Процентная_ставка(NUMERIC(10))

Ключевые особенности:

-индекс по Статус_кредита для быстрого поиска

Таблица «Платеж»

ID_PK – Уникальный идентификатор, первичный ключ, автоувеличение

ФИО(TEXT(50))

Дата_платежа(DATE)

Вид_платежа(TEXT(15))

Сумма_платежа(NUMERIC(10))

Сумма_кредита(NUMERIC(10))

Остаток_долга(NUMERIC(15))

Статус_кредита(TEXT(10))

Цель_кредита(ТЕХТ(20))

Ключевые особенности:

-индекс по Цель_кредита для быстрого поиска

3. Разработка и реализация АИС

3.1. Создание SQL-запросов

SQL-запросы являются основным механизмом взаимодействия с реляционными базами данных. Они позволяют эффективно управлять данными, обеспечивая их хранение, извлечение, модификацию и удаление.

Результатом запроса является представление — виртуальная таблица, представляющая собой поименованный запрос, который будет подставлен как подзапрос при использовании представления. В отличие от обычных таблиц реляционных баз данных, представление не является самостоятельной частью набора данных, хранящегося в базе.

1.1.1 Запрос создания таблицы (CREATE TABLE):

Запрос для создания новой таблицы Договор.

```
CREATE TABLE Договор (ID_Кредита INTEGER, ID_Клиента
INTEGER, ID_Сотрудника TEXT, ID INTEGER PRIMARY KEY UNIQUE,
Фамилия TEXT (20) NOT NULL,
Имя TEXT (20) NOT NULL,
Отчество TEXT (20) NOT NULL,
Дата_рождения TEXT (10) NOT NULL,
Серия_паспорта INTEGER (4) NOT NULL,
Номер_паспорта INTEGER (6) NOT NULL,
ИНН INTEGER (8) NOT NULL,
Адрес_проживания TEXT (20) NOT NULL,
Номер_телефона INTEGER (11) NOT NULL,
Банк TEXT (15) NOT NULL,
Сумма_кредита INTEGER (10) NOT NULL,
Срок_кредита TEXT (15) NOT NULL,
Процентная_ставка TEXT (15) NOT NULL,
Цель_кредита TEXT (25) NOT NULL,
```

FOREIGN KEY (ID_Сотрудника) REFERENCES Сотрудник (ID) ON DELETE CASCADE ON UPDATE CASCADE,

FOREIGN KEY (ID_Клиента) REFERENCES Клиент (ID) ON DELETE CASCADE

ON UPDATE CASCADE,

FOREIGN KEY (ID_Кредита) REFERENCES Кредит (ID) ON DELETE CASCADE

ON UPDATE CASCADE);

CREATE TABLE Договор - Эта команда создаёт новую таблицу с названием Договор.

ID INTEGER PRIMARY KEY UNIQUE – создается столбец ID с типом данных INTEGER со значением, являющийся первичным ключом PRIMARY KEY.

Фамилия TEXT (20) NOT NULL – команды данного типа создают столбцы с указанными типами данных.

FOREIGN KEY (ID_Сотрудника) REFERENCES Сотрудник (ID) ON DELETE CASCADE ON UPDATE CASCADE – данный тип команд связывает столбцы через внешний ключи, в данном случае столбец ID_Сотрудника является внешним ключом к таблице Сотрудник, где первичным ключом является столбец ID.

1.1.2 Запросы для вывода данных из таблиц (SELECT)

1) Запрос для вывода данных из таблицы Клиент

SELECT * FROM Клиент

Данный запрос выбирает все столбцы в таблице Клиент с помощью команды FROM.

Результат выполнения показан на рисунке 13.

	Фамилия	Имя	Отчество	Дата_рождения	Серия_паспорта	Номер_паспорта	ИНН	Адрес_проживания	Номер_телефона
►	Рысев	Андрей	Сергеевич	10.08.2004	1318	958956	28641904	Муром ул.Куйбы...	89086814394
	Болотин	Даниил	Павлович	12.07.2004	4561	783210	643910	Меленки ул.Гри...	89956122815
	Степыко	Денис	Владимирович	15.04.2004	3851	943170	739051	Выкса ул.Пушки...	89946721953
	Назаренко	Мария	Ивановна	09.09.2004	5328	904629	309531	Краснода ул.Кр...	89974532901
*									

Рис. 13 – Результат выполнения запроса на таблице Клиент

2) Запрос для вывода данных из таблицы Договор, а также вставка полученный данных с таблицы Кредит и Клиент.

```
SELECT * FROM Договор

Триггер (Клиент):

CREATE TRIGGER Триггер_Клиент
AFTER INSERT
ON Договор
WHEN NEW.ИНН IS NOT NULL
BEGIN

INSERT INTO Клиент (Фамилия, Имя, Отчество, Дата_рождения,
Серия_паспорта, Номер_паспорта, ИНН, Адрес_проживания, Номер_телефона)
VALUES (NEW.Фамилия, NEW.Имя, NEW.Отчество, NEW.Дата_рождения,
NEW.Серия_паспорта, NEW.Номер_паспорта, NEW.ИНН,
NEW.Адрес_проживания, NEW.Номер_телефона);

UPDATE Договор
SET ID_Клиента = (SELECT MAX(ID) FROM Клиент)
WHERE ROWID = NEW.ROWID;

END;
```

Этот триггер выполняет действия после вставки новой записи в таблицу Договор. Рассмотрим каждое слово:

1. CREATE TRIGGER Триггер_Клиент – создает триггер с именем Триггер_Клиент.

2. AFTER INSERT – триггер срабатывает после вставки новой строки в таблицу.

3. ON Договор – триггер применяется к таблице Договор.

4. WHEN NEW.ИНН IS NOT NULL – триггер выполняется только если значение ИНН в новой записи не пустое.

5. BEGIN – начало блока действий, которые выполняются при срабатывании триггера.

В теле триггера:

1. INSERT INTO Клиент – вставляет новую запись в таблицу Клиент.
2. (Фамилия, Имя, Отчество, Дата_рождения, Серия_паспорта, Номер_паспорта, ИНН, Адрес_проживания, Номер_телефона) – указывает столбцы, в которые будут вставлены значения.
3. VALUES (NEW.Фамилия, NEW.Имя, NEW.Отчество, NEW.Дата_рождения, NEW.Серия_паспорта, NEW.Номер_паспорта, NEW.ИНН, NEW.Адрес_проживания, NEW.Номер_телефона) – значения для вставки берутся из только что добавленной записи в таблице Договор.
4. UPDATE Договор – обновляется таблица Договор.
5. SET ID_Клиента = (SELECT MAX(ID) FROM Клиент) – в столбец ID_Клиента добавляется максимальное значение ID из таблицы Клиент, которое соответствует только что добавленной записи.
6. WHERE ROWID = NEW.ROWID – обновляется только та запись в таблице Договор, которая была вставлена (определяется по идентификатору строки ROWID).
7. END – завершение блока действий.

Таким образом, этот триггер выполняет вставку данных в таблицу Клиент, а затем обновляет таблицу Договор, добавляя ссылку на только что добавленного клиента.

Аналогично этому триггеру был создан триггер для кредита:

Триггер (Кредит):

```
CREATE TRIGGER Триггер_Кредит
AFTER INSERT
ON Договор
WHEN NEW.Сумма_кредита IS NOT NULL
BEGIN
```

```

INSERT INTO Кредит (Банк, Цель_кредита, Срок_кредита, Сумма_кредита,
Процентная_ставка, Дата_выдачи_кредита)
VALUES      (NEW.Банк,      NEW.Цель_кредита,      NEW.Срок_кредита,
NEW.Сумма_кредита, NEW.Процентная_ставка, DATE('now') );

UPDATE Договор
SET ID_Кредита = (SELECT MAX(ID) FROM Кредит)
WHERE ROWID = NEW.ROWID;

END;

```

Этот триггер выполняет действия после вставки новой записи в таблицу Договор и связан с таблицей Кредит. Рассмотрим каждое слово:

1. CREATE TRIGGER Триггер_Кредит – создает триггер с именем Триггер_Кредит.
2. AFTER INSERT – триггер срабатывает после вставки новой строки в таблицу.
3. ON Договор – триггер применяется к таблице Договор.
4. WHEN NEW.Сумма_кредита IS NOT NULL – триггер выполняется только если значение Сумма_кредита в новой записи не пустое.
5. BEGIN – начало блока действий, которые выполняются при срабатывании триггера.

В теле триггера:

1. INSERT INTO Кредит – вставляется новая запись в таблицу Кредит.
2. (Банк, Цель_кредита, Срок_кредита, Сумма_кредита, Процентная_ставка, Дата_выдачи_кредита) – указывает столбцы, в которые будут вставлены значения.
3. VALUES (NEW.Банк, NEW.Цель_кредита, NEW.Срок_кредита, NEW.Сумма_кредита, NEW.Процентная_ставка, DATE('now')) – значения для вставки берутся из только что добавленной записи в таблице Договор. Для

столбца Дата_выдачи_кредита используется функция DATE('now'), которая вставляет текущую дату.

4. UPDATE Договор – обновляется таблица Договор.

5. SET ID_Кредита = (SELECT MAX(ID) FROM Кредит) – в столбец ID_Кредита таблицы Договор добавляется максимальное значение ID из таблицы Кредит, которое соответствует только что добавленной записи.

6. WHERE ROWID = NEW.ROWID – обновляется только та запись в таблице Договор, которая была вставлена (определяется по идентификатору строки ROWID).

7. END – завершение блока действий.

Таким образом, этот триггер вставляет данные о кредите в таблицу Кредит (с использованием данных из записи в таблице Договор) и затем обновляет таблицу Договор, добавляя ссылку на только что добавленный кредит.

Результат выполнения показан на рисунке 14, 15, 16.

Клиенты Договор Кредит Платеж											
	Фамилия	Имя	Отчество	Дата_рождения	Серия_паспорта	Номер_паспорта	ИНН	Адрес_проживани	Номер_телефона	Банк	Сумма_кредит
►	Рысев	Андрей	Сергеевич	10.08.2004	1318	958956	28641904	Муром ул. Куйбы...	89086814394	Сбербанк	145000
	Болотин	Даниил	Павлович	12.07.2004	4561	783210	643910	Меленки ул. Гри...	89956122815	Альфа-банк	15000000
	Степко	Денис	Владимирович	15.04.2004	3851	943170	739051	Выкса ул. Пушки...	89946721953	Сбербанк	150000
	Назаренко	Мария	Ивановна	09.09.2004	5328	904629	309531	Краснода ул. Кр...	89974532901	ВТБ	50000
*											

Рис. 14 – Результат выполнения запроса на таблице Договор

Клиенты Договор Кредит Платеж							
	Банк	Цель_кредита	Срок_кредита	Статус_кредита	Дата_выдачи_кре	Сумма_кредита	Процентная_стави
►	Сбербанк	Личные цели	12 месяцев	Активен	2024-12-25	145000	5%
	Альфа-банк	Автокредит	48 месяцев	Активен	2024-12-25	15000000	10%
	Сбербанк	Потребительск...	24 месяца	Активен	2024-12-25	150000	5%
	ВТБ	Жилищные цели	12 месяцев	Активен	2024-12-25	50000	5%
*							

Рис. 15 – Результат выполнения запроса на таблице Кредит

Клиенты	Договор	Кредит	Платеж						
	Фамилия	Имя	Отчество	Дата_рождения	Серия_паспорта	Номер_паспорта	ИНН	Адрес_проживани	Номер_телефона
▶	Рысев	Андрей	Сергеевич	10.08.2004	2024	958956	28641904	Муром	89086814394
	Болотин	Даниил	Павлович	12.07.2004	4561	783210	643910	Меленки ул.Гри...	89956122815
	Степыко	Денис	Владимирович	15.04.2004	3851	943170	739051	Выкса ул.Пушки...	89946721953
	Назаренко	Мария	Ивановна	09.09.2004	5328	904629	309531	Краснода ул.Кр...	89974532901
*									

Рис. 16 – Результат выполнения запроса на таблице Кредит

1.1.3 Запрос для вставки в таблицу (INSERT):

Вставка в таблицу Договор при оформлении кредита:

“INSERT INTO Договор (Фамилия, Имя, Отчество, Дата_рождения, Серия_паспорта, Номер_паспорта, ИНН, Адрес_проживания, Номер_телефона, Банк, Сумма_кредита, Срок_кредита, Процентная_ставка, Цель_кредита) " +
 "VALUES (@LastName, @FirstName, @MiddleName, @BirthDate, @PassportSeries, @PassportNumber, @INN, @Address, @PhoneNumber, @BankName, @CreditAmount, @CreditTerm, @AnnualRate, @CreditPurpose)";
 INSERT INTO Договор (Фамилия, Имя, Отчество, Дата_рождения, Серия_паспорта, Номер_паспорта, ИНН, Адрес_проживания, Номер_телефона, Банк, Сумма_кредита, Срок_кредита, Процентная_ставка, Цель_кредита) – эта часть запроса перечисляет столбцы, в которые будут вставляться данные, и задает сам факт вставки данных.

"VALUES (@LastName, @FirstName, @MiddleName, @BirthDate, @PassportSeries, @PassportNumber, @INN, @Address, @PhoneNumber, @BankName, @CreditAmount, @CreditTerm, @AnnualRate, @CreditPurpose)"; – данная часть перечисляет значения, которые будут вставлены в соответствующие столбцы. При выполнении запроса все переменные будут заменены на конкретные значения.

1.1.4 Запрос для обновления записей (UPDATE)

Обновление данных в таблице Клиент:

"UPDATE Клиент SET Фамилия = @surname, Имя = @name, Отчество = @lastName, ИНН = @INN, Дата_рождения = @data_roda, Серия_паспорта = @seria_pasporta, Номер_паспорта = @number_pasporta, Адрес_проживания = @address, Номер_телефона = @number_phone, Дата_регистрации = @data_reg WHERE ID = @id";

UPDATE Клиент SET – эта часть запроса указывает на факт обновления данных в таблице Клиент.

SФамилия = @surname, Имя = @name, Отчество = @lastName, ИНН = @INN, Дата_рождения = @data_roda, Серия_паспорта = @seria_pasporta, Номер_паспорта = @number_pasporta, Адрес_проживания = @address, Номер_телефона = @number_phone, Дата_регистрации = @data_reg WHERE ID = @id"; – данная часть показывает столбец и новое значение этого столбца. При выполнении запроса, все значения будут заменены на конкретные.

1.1.5 Запрос для удаления записей (DELETE)

Удаление данных из таблицы Клиент:

"DELETE FROM Клиент WHERE ID = @id";

DELETE FROM Клиент – эта часть запроса указывает на факт удаления данных из таблицы Клиент.

WHERE ID=@ID – данная часть показывает, что будет произведено удаление строки с конкретным значением поля ID. При выполнении запроса значение будет заменено на конкретное.

3.2. Руководство пользователя

При запуске приложения первым шагом для пользователя служит окно авторизации, показанное на рисунке 17. Оно служит для идентификации и аутентификации пользователей.

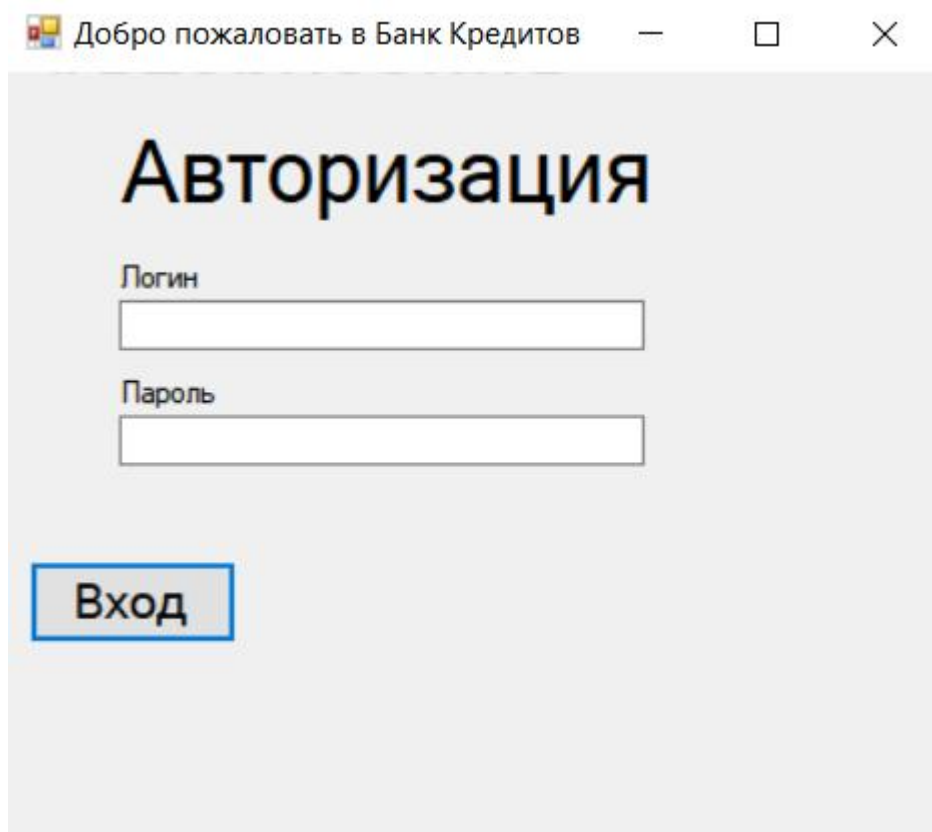
The image shows a screenshot of a software window titled "Добро пожаловать в Банк Кредитов". Inside the window, the word "Авторизация" is displayed in a large, bold font. Below it, there are two input fields: the first is labeled "Логин" and the second is labeled "Пароль". At the bottom left of the window, there is a button labeled "Вход". The window has standard Windows-style controls (minimize, maximize, close) in the top right corner.

Рис. 17 – Окно авторизации

В окне авторизации пользователь вводит логин и пароль от своей учетной записи, что показано на рисунках 18 и 19. Если логин и пароль правильные, то он попадает в рабочее окно, что показано рисунках 20, 21 и 22.

Добро пожаловать в Банк Кредитов

Авторизация

Логин

Пароль

Вход

Рис. 18 – Ввод данных от учетной записи клиента

Добро пожаловать в Бан...

Авторизация

Логин

Пароль

Вход

Рис. 19 – Ввод данных от учетной записи администратора

Пользовательский интерфейс

Фамилия	Имя	Отчество	Дата_рождения	Серия_паспорта	Номер_паспорта	ИНН	Адрес_проживания	Номер_телефона	Банк	Сумма_кредита	Срок_кредита
Рысев	Андрей	Сергеевич	10.08.2004	1318	958956	28641904	Муром ул.Куйбы...	89086814394	Т-банк	145000	12 месяцев
Болотин	Даниил	Павлович	12.07.2004	4561	783210	643910	Меленки ул.Гри...	89956122815	Альфа-банк	15000000	48 месяцев
Степко	Денис	Владимирович	15.04.2004	3851	943170	739051	Выкса ул.Пушки...	89946721953	Сбербанк	150000	24 месяца
Назаренко	Мария	Ивановна	09.09.2004	5328	904629	309531	Краснода ул.Кр...	89974532901	ВТБ	50000	12 месяцев

Фамилия Имя Отчество

Дата рождения Серия паспорта Номер паспорта

ИНН Адрес Банк

Номер телефона Сумма кредита Срок кредита

Процент ставка Цель кредита

Оформить

Изменить

Совершить платеж

Поиск

Искать Сброс

Рис. 20 – Рабочее окно клиента

Платеж

ФИО	Цель_кредита	Дата_платежа	Вид_платежа	Сумма_кредита	С...
Рысев Андрей С...	Личные цели	25-12-2024	Перевод	145000	75
Рысев Андрей С...	Личные цели	25-12-2024	Перевод	0	35
Рысев Андрей С...	Личные цели	25-12-2024	Перевод	0	35
Степко Денис ...	Потребительск...	25-12-2024	Наличные	150000	15
Назаренко Мар...	Жилищные цели	25-12-2024	Наличные	50000	25

ФИО Цель кредита Дата платежа

Вид платежа Сумма кредита Сумма платежа

Остаток суммы Статус кредита

Совершить платеж

Поиск

Искать Сброс

Рис. 21 – Рабочее окно клиента (Совершение платежа)

Интерфейс администратора

Обновить Выход

Клиенты Договор Кредит Платеж

	Фамилия	Имя	Отчество	Дата_рождения	Серия_паспорта	Номер_паспорта	ИНН
▶	Рысев	Андрей	Сергеевич	10.08.2004	2024	958956	28641904
	Болотин	Даниил	Павлович	12.07.2004	4561	783210	643910
	Степко	Денис	Владимирович	15.04.2004	3851	943170	739051
	Назаренко	Мария	Ивановна	09.09.2004	5328	904629	309531
*							

Фамилия Серия паспорта Номер телефона

Имя Номер паспорта Дата регистрации

Отчество ИНН

Дата рождения Адрес проживания

Добавить Изменить Удалить

Поиск

Фамилия

Найти Сброс

Рис. 22 – Рабочее окно администратора

Рабочее окно менеджера или окно аренды, показанное на рисунке 18 имеет следующие функции:

- 1) Текстовые поля для ввода данных клиента, желающего оформить кредит
- 2) Кнопка «Оформить» для записи данных нового клиента в базу и оформления договора об кредите
- 3) Поле поиска по любой из характеристик
- 4) Кнопка «Совершить платеж» для совершения платежа, которая перенаправляет клиента в другое диалоговое окно.
- 5) В диалоговом окне платежа кнопка «Совершить платеж», которая записывает данные о платеже в базу данных.

Форма администратора, представленная на рисунке 22 состоит из нескольких страниц для каждой из таблиц в базе. Она имеет следующие функции:

- 1) Возможность просмотра необходимых данных, они показаны на рисунках 23 – 26

Интерфейс администратора

Обновить Выход

Клиенты Договор Кредит Платеж

	Фамилия	Имя	Отчество	Дата_рождения	Серия_паспорта	Номер_паспорта	ИН
▶	Рысев	Андрей	Сергеевич	10.08.2004	2024	958956	286
	Болотин	Даниил	Павлович	12.07.2004	4561	783210	643
	Степыко	Денис	Владимирович	15.04.2004	3851	943170	739
	Назаренко	Мария	Ивановна	09.09.2004	5328	904629	309
*							

Фамилия Серия паспорта Номер телефона
Имя Номер паспорта Дата регистрации
Отчество ИНН
Дата рождения Адрес проживания
Добавить Изменить Удалить

Поиск
Фамилия
Найти Сброс

Рис. 23 – Данные из таблицы Клиенты

Интерфейс администратора

Обновить Выход

Клиенты Договор Кредит Платеж

	Фамилия	Имя	Отчество	Дата_рождения	Серия_паспорта	Номер_пас
▶	Рысев	Андрей	Сергеевич	10.08.2004	2024	958956
	Болотин	Даниил	Павлович	12.07.2004	4561	783210
	Степыко	Денис	Владимирович	15.04.2004	3851	943170
	Назаренко	Мария	Ивановна	09.09.2004	5328	904629
*						

Фамилия Серия паспорта Номер телефона Процентная ставка
Имя Номер паспорта Банк Цель кредита
Отчество ИНН Сумма кредита
Дата рождения Адрес Срок кредита
Добавить Изменить Удалить

Поиск
Фамилия
Найти Сброс

Рис. 24 – Данные из таблицы Договор

Интерфейс администратора

Обновить Выход

Клиенты Договор Кредит Платеж

	Банк	Цель_кредита	Срок_кредита	Статус_кредита	Дата_выдачи_кре	Сумма_кредита	Проц
▶	Сбербанк	Личные цели	12 месяцев	Активен	2024-12-25	145000	5.3%
	Альфа-банк	Автокредит	48 месяцев	Активен	2024-12-25	15000000	10%
	Сбербанк	Потребительск...	24 месяца	Активен	2024-12-25	150000	5%
	ВТБ	Жилищные цели	12 месяцев	Активен	2024-12-25	50000	5%
*							

Цель кредита
Личные цели ▾
Срок кредита
12 месяцев ▾
Статус кредита
Активен ▾

Сумма кредита
145000
Процентная ставка
5.3% ▾
Дата выдачи кредита
2024-12-25

Добавить Изменить Удалить

Поиск

Банк ▾
Найти
Сброс

Рис. 25 – Данные из таблицы Кредит

Интерфейс администратора

Обновить Выход

Клиенты Договор Кредит Платеж

	ФИО	Цель_кредита	Дата_платежа	Вид_платежа	Сумма_кредита	Сумма_платежа	Остаток_долг
▶	Рысев Андрей С...	Личные цели	25-12-2024	Перевод	145000	75000	70000
	Рысев Андрей С...	Личные цели	25-12-2024	Перевод	0	35000	35000
	Рысев Андрей С...	Личные цели	25-12-2024	Перевод	0	35000	0
	Степыко Денис ...	Потребительск...	25-12-2024	Наличные	150000	150000	0
	Назаренко Мар...	Жилищные цели	25-12-2024	Наличные	50000	25000	25000
*							

ФИО
Рысев Андрей С... ▾
Вид платежа
Перевод
Остаток суммы
0

Цель кредита
Личные цели
Сумма кредита
145000
Сумма платежа
75000
Статус кредита

Дата платежа
25-12-2024

Добавить Изменить Удалить

Поиск

ФИО ▾
Искать
Сброс

Рис. 26 – Данные из таблицы Платеж

2) Каждая из страниц имеет текстовые поля для ввода данных, а также кнопки для их добавления, обновления или удаления. Для добавления данных в таблицу необходимо заполнить все поля, как показано на рисунке 26 и нажать на кнопку. Результат показан на рисунке 27.

Фамилия	Имя	Отчество
Голуби	Павел	Павлович
Дата рождения	Серия паспорта	Номер паспорта
11.09.2001	6538	943780
ИНН	Адрес	Банк
56890321	Муром. ул.Совет	Альфа-банк ▾
Номер телефона	Сумма кредита	Срок кредита
89956122815	200000	36 месяцев ▾
Процент.ставка	Цель кредита	
6.2% ▾	Личные цели ▾	
Оформить		
Изменить		
Совершить платеж		

Рис. 27 – Заполнение полей

	Фамилия	Имя	Отчество	Дата_рождения	Серия_паспорта	Номер_паспорта	ИНН	Адрес_проживания	Номер_телефона	Банк	Сумма_кредита	Срок_кредита
»	Голуби	Павел	Павлович	11.09.2001	6538	943780	56890321	Муром. ул.Сове...	89956122815	Альфа-банк	200000	36 месяцев

Рис. 28 – Запись добавлена в таблицу

3) Для изменения данных необходимо выбрать запись, все данные перенесутся в текстовые поля, изменить данные и нажать на кнопку. Результат показан на рисунках 29-30.

Фамилия	Имя	Отчество
Голуби	Павел	Павлович
Дата рождения	Серия паспорта	Номер паспорта
11.09.2001	4289	653901
ИНН	Адрес	Банк
56890321	Муром. ул.Совет	Альфа-банк ▾
Номер телефона	Сумма кредита	Срок кредита
89956122815	200000	36 месяцев ▾
Процент.ставка	Цель кредита	
6.2% ▾	Личные цели ▾	
Оформить		
Изменить		
Совершить платеж		

Рис. 29 – Изменение данных

	Фамилия	Имя	Отчество	Дата_рождения	Серия_паспорта	Номер_паспорта	ИНН	Адрес_проживани
►	Голуби	Павел	Павлович	11.09.2001	4289	653901	56890321	Муром. ул.Сове...
*								

Успех

Данные успешно обновлены.

ОК

Рис. 30 – Запись обновляется

4) Для удаления данных необходимо также выбрать запись и нажать на кнопку удаления. Результат представлен на рисунках 31.

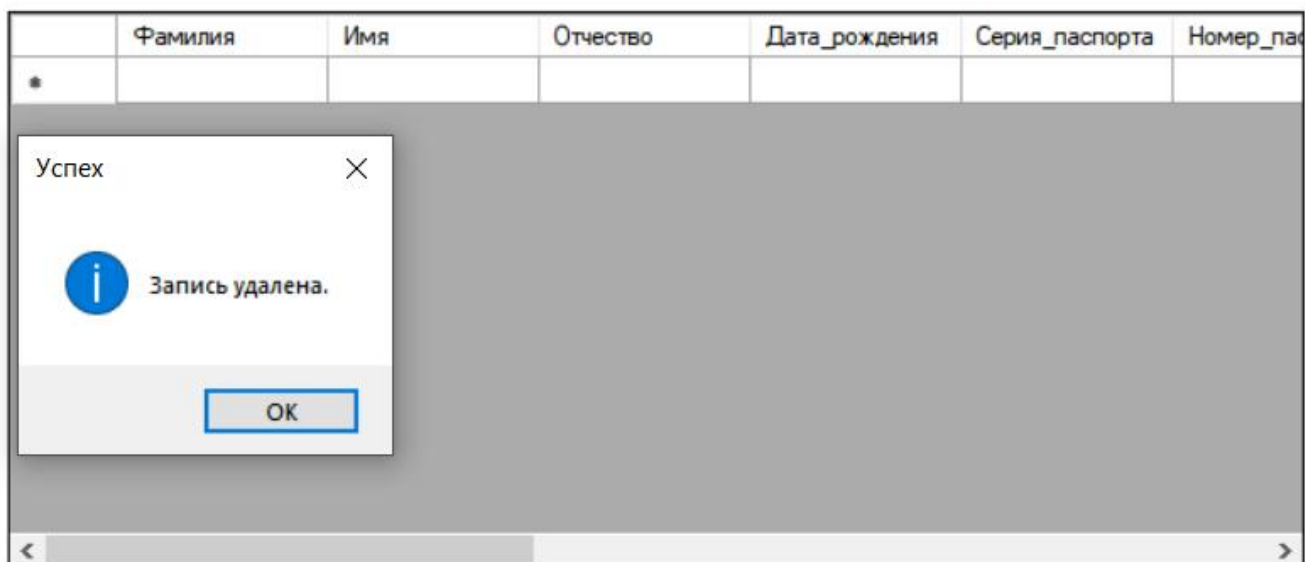


Рис. 31 – Запись удаления

5) Также каждая из страниц обладает возможностью поиска данных по параметрам. Для этого необходимо выбрать параметр из выпадающего списка, ввести нужное значение в текстовое поле и нажать кнопку поиска. Результаты показаны на рисунках 32-33.

Поиск

Серия_паспорт ▼

Найти

Сброс

Рис. 32 – Выбор параметра

	Фамилия	Имя	Отчество	Дата_рождения	Серия_паспорта	Номер_паспорта
▶	Рысев	Андрей	Сергеевич	10.08.2004	1318	958956
*						

Фамилия	Серия паспорта	Номер телефона	Процентная ставка
Имя	Номер паспорта	Банк	Цель кредита
Отчество	ИНН	Сумма кредита	
Дата рождения	Адрес	Срок кредита	
Добавить Изменить Удалить			

Поиск

Серия_паспорт ▼

Найти

Сброс

Рис. 33 – Результат поиска

6) Также после оформления кредита клиент может погасить его, нажав на кнопку «Совершить платеж» и будет выводиться диалоговое окно с подтверждением. Результат показан на рисунке 41.

The image shows a software interface for credit management. On the left, there is a table with columns: 'Серия_паспорта', 'Номер_паспорта', and 'ИНН'. It contains several rows of data. Overlaid on this table is a 'Подтверждение' (Confirmation) dialog box with a question mark icon and the text 'Вы хотите совершить платеж?' (Do you want to make a payment?). Below the question are 'Да' (Yes) and 'Нет' (No) buttons. To the right of the dialog is a form for entering credit details. It includes fields for 'Фамилия', 'Имя', 'Отчество', 'Дата рождения', 'Серия паспорта', 'Номер паспорта', 'ИНН', 'Адрес', 'Банк', 'Номер телефона', 'Сумма кредита', 'Срок кредита', 'Процент. ставка', and 'Цель кредита'. At the bottom of the form are buttons for 'Оформить', 'Изменить', and 'Совершить платеж' (which is highlighted with a blue border).

Рис. 34 – Диалоговое окно с подтверждением

Далее клиент может совершить платеж, выбрав в поле «ФИО» свою Фамилию, Имя и Отчество. Затем он указывает сумма платежа и нажимает кнопку «Оплатить». Результаты показаны на рисунке 42.

The image shows a window titled 'Платеж' (Payment). It contains a table with columns: 'ФИО', 'Цель_кредита', 'Дата_платежа', 'Вид_платежа', 'Сумма_кредита', and 'С'. The table lists several payments, including those by 'Рысев Андрей С...' and 'Гусев Максим...'. To the right of the table is a form for making a payment. It includes fields for 'ФИО', 'Цель кредита', 'Дата платежа', 'Вид платежа', 'Сумма кредита', 'Сумма платежа', 'Остаток суммы', and 'Статус кредита'. At the bottom of the form is a button labeled 'Совершить платеж'.

Рис. 35 – Данные из таблицы Платеж

7) Также на Form3 находится меню с 2 кнопками: «Обновить» и «Выход». Первая кнопка обновляет данные в базе данных в случае необходимости, вторая кнопка позволяет выйти из программы. Результат показан на рисунке 34.

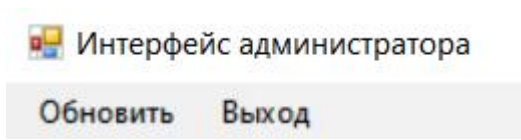


Рис. 36 – Меню настроек

3.3. Руководство программиста

3.3.1 Введение

Приложение разработано на языке программирования C# в среде Visual Studio с использованием СУБД SQLite. Программа включает в себя несколько форм: форма авторизации, рабочая форма администратора, рабочая форма менеджера по аренде и форма архива.

3.3.2 Установка и настройка

Необходимые компоненты:

- Visual Studio 2022 или новее
- .NET Framework 4.7.2 или новее
- СУБД SQLite

Установка:

- 1) Откройте Visual Studio
- 2) Запустите проект
- 3) Установите пакет SQLite через пункт управление NuGet пакетами

3.3.3 Структура приложения

- Проект состоит из следующих файлов:
- Form1.cs – Форма авторизации
- Form2.cs – Форма клиента для оформления кредита
- Form3.cs – Форма администратора
- Form4.cs – Форма совершения платежа (для клиента)

3.3.4 Описание форм

1) Form1.cs

- Конструктор Form1: инициализирует форму.
- Метод button1_Click: обрабатывает нажатие кнопки входа, проверяет введенные в текстовые поля логин и пароль и открывает соответствующую форму.
- Метод textBox2.PasswordChar: скрывает ввод пароля с помощью *.

2) Form2.cs

- Конструктор Form2: инициализирует форму, устанавливает соединение с базой данных, заполняет comboBox2 данными о сроке кредита, comboBox3 – данными о процентной ставке, comboBox4 - данными о цели кредита.
- Метод LoadDogovorData: выполняет SQL-запрос и загружает данные об имеющихся договорах и выводит их в dataGridView1.
- Метод button1_Click: обрабатывает нажатие кнопки кредитования, проверяет заполненность текстовых полей о данных клиента выполняет SQL-запрос, заносит запись в таблицу Договор, а также с помощью триггеров заполняет таблицы Кредит и Клиент и в таблицу Договор,
- Метод textBox1_Validating: проверяет корректность ввода фамилии в textBox1.
- Метод textBox2_Validating: проверяет корректность ввода имени в textBox2.
- Метод textBox3_Validating: проверяет корректность ввода отчества в textBox3.
- Метод textBox4_Validating: проверяет корректность ввода даты рождения в textBox4.
- Метод textBox5_Validating: проверяет корректность ввода серии паспорта в textBox5.
- Метод textBox6_Validating: проверяет корректность ввода номера паспорта в textBox6.
- Метод textBox7_Validating: проверяет корректность ввода ИНН в textBox7.
- Метод textBox9_Validating: проверяет корректность ввода номера телефона в textBox9.
- Метод IsValidName: проверяет выбрана ли строка.
- Метод IsNumeric: проверяет, что введено числовое значение, используется при проверке серии, номера паспорта и номера телефона.
- Метод button1_Click: обрабатывает нажатие кнопки поиска, выполняет SQL-запрос и добавляет данные таблицы в dataGridView1.

- Метод `button3_Click`: обрабатывает нажатие кнопки поиска, выполняет SQL-запрос и изменяет данные таблицы в `dataGridView1`.
- Метод `dataGridView1_SelectionChanged`: обрабатывает выбор строки в `dataGridView1` и переносит данные в `textbox` и `combobox`.
- Метод `ClearInputs`: очищает `textbox` и `combobox` после выполнения добавления, изменения данных
- Метод `LoadComboBox1FromBank`: выполняет SQL_запрос и заполняет `combobox` заданными значениями.
- Метод `button2_Click`: обрабатывает нажатие кнопки поиска, выполняет SQL-запрос и выводит результат в `dataGridView1`.
- Метод `button5_Click`: обрабатывает нажатие кнопки и очищает поле поиска в `dataGridView1`.
- Метод `LoadColumnNames`: заполняет `combobox` наименованиями из `dataGridView1`.

3) Form3.cs

- Конструктор `Form3`: инициализирует форму, устанавливает соединение с базой данных и заполняет `ComboBox2`, `ComboBox3`, `ComboBox4`, `ComboBox9`, `ComboBox10`, `ComboBox11`, `ComboBox13` необходимыми данными.
- Метод `LoadClientData`: выполняет SQL-запрос и заполняет `dataGridView1` данными о клиентах.
- Метод `LoadDogovorData`: выполняет SQL-запрос и заполняет `dataGridView2` данными об договорах.
- Метод `LoadKreditData`: выполняет SQL-запрос и заполняет `dataGridView3` данными о кредитах.
- Метод `LoadPlatechData`: выполняет SQL_запрос и заполняет `dataGridView4` данными о платежах.
- Метод `button1_Click`: обрабатывает нажатие кнопки добавления записей в `dataGridView1`, выполняет SQL-запрос и добавляет запись в зависимости от введенных данных.

- Метод `button2_Click`: обрабатывает нажатие кнопки обновления записей в `dataGridView1`, выполняет SQL-запрос и обновляет запись в зависимости от введенных данных.
- Метод `dataGridView1_CellClick`: обрабатывает выбор строки в `dataGridView1` и переносит данные в `textbox`
- Метод `button4_Click`: обрабатывает нажатие кнопки удаления записей из `dataGridView1`, выполняет SQL-запрос и удаляет запись.
- Метод `InsertKlientDataToDatabase`: выполняет SQL_запрос и добавляет данные в `dataGridView1`.
- Метод `UpdateKlientDataInDatabase`: выполняет SQL_запрос и изменяет данные в `dataGridView1`.
- Метод `DeleteKlientDataFromDatabase`: выполняет SQL_запрос и удаляет выбранную строку в `dataGridView1`.
- Метод `button5_Click`: обрабатывает нажатие кнопки добавления записей в `dataGridView2`, выполняет SQL-запрос и добавляет запись в зависимости от введенных данных.
- Метод `button6_Click`: обрабатывает нажатие кнопки обновления записей в `dataGridView2`, выполняет SQL-запрос и обновляет запись в зависимости от введенных данных.
- Метод `dataGridView2_CellClick`: обрабатывает выбор строки в `dataGridView2` и переносит данные в `textbox` и `combobox`
- Метод `button8_Click`: обрабатывает нажатие кнопки удаления записей из `dataGridView2`, выполняет SQL-запрос и удаляет запись.
- Метод `button3_Click_1` обрабатывает нажатие кнопки добавления записей в `dataGridView3`, выполняет SQL-запрос и добавляет запись в зависимости от введенных данных.
- Метод `button7_Click_1`: обрабатывает нажатие кнопки обновления записей в `dataGridView3`, выполняет SQL-запрос и обновляет запись в зависимости от введенных данных.

- Метод `dataGridView3_SelectionChanged`: обрабатывает выбор строки в `dataGridView3` и переносит данные в `textbox` и `combobox`
- Метод `button9_Click`: обрабатывает нажатие кнопки удаления записей из `dataGridView3`, выполняет SQL-запрос и удаляет запись.
- Метод `InsertKreditDataToDatabase`: выполняет SQL_запрос и добавляет данные в `dataGridView3`.
- Метод `UpdateKreditDataInDatabase`: выполняет SQL_запрос и обновляет данные в `dataGridView3`.
- Метод `DeleteKreditDataFromDatabase`: выполняет SQL_запрос и удаляет выбранную строку в `dataGridView3`.
- Метод `button13_Click_1`: обрабатывает нажатие кнопки добавления записей в `dataGridView4`, выполняет SQL-запрос и добавляет запись в зависимости от введенных данных.
- Метод `button14_Click_1`: обрабатывает нажатие кнопки обновления записей в `dataGridView4`, выполняет SQL-запрос и обновляет запись в зависимости от введенных данных.
- Метод `dataGridView4_SelectionChanged`: обрабатывает выбор строки в `dataGridView4` и переносит данные в `textbox` и `combobox`
- Метод `button15_Click_1`: обрабатывает нажатие кнопки удаления записей из `dataGridView4`, выполняет SQL-запрос и удаляет запись.
- Метод `InsertPlatechDataToDatabase`: выполняет SQL_запрос и добавляет данные в `dataGridView4`.
- Метод `UpdatePlatechDataInDatabase`: выполняет SQL_запрос и обновляет данные в `dataGridView4`.
- Метод `DeletePlatechDataFromDatabase`: выполняет SQL_запрос и удаляет выбранную строку в `dataGridView4`.
- Метод `comboBox12_SelectedIndexChanged`: выполняет SQL_запрос и заполняет `combobox` заданными значениями.

- Метод button17_Click: обрабатывает нажатие кнопки добавления записей в dataGridview5, выполняет SQL-запрос и добавляет запись в зависимости от введенных данных.
- Метод LoadClientsToComboBox: выполняет SQL_запрос и заполняет combobox заданным значением.
- Метод InitializeFormComponents: создает компонент «Меню» на Form3, также создает элементы меню «Обновить» и «Выход» и задает размер.
- Метод RefreshMenuItem_Click: обрабатывает нажатие кнопки обновление данных в БД.
- Метод ExitMenuItem_Click: обрабатывает нажатие кнопки выхода из Form3.
- Метод LoadSearchCriteria: выполняет SQL_запрос для нормальной работы поиска, заполняет combobox наименованиями столбцов из таблиц БД.
- Метод button10_Click: обрабатывает нажатие кнопки поиска, выполняет SQL-запрос и выводит результат в dataGridview1.
- Метод button11_Click: обрабатывает нажатие кнопки поиска, выполняет SQL-запрос и выводит результат в dataGridview2.
- Метод button12_Click: обрабатывает нажатие кнопки поиска, выполняет SQL-запрос и выводит результат в dataGridview3.
- Метод button16_Click: обрабатывает нажатие кнопки поиска, выполняет SQL-запрос и выводит результат в dataGridview4.
- Метод ClearInputs: очищает textbox и combobox после выполнения добавления, изменения данных
- Метод IsValidINN: проверяет корректность ИНН.
- Метод IsValidSeriaPasporta: проверяет корректность Серия паспорта.
- Метод IsValidNumberPasporta: проверяет корректность Номера паспорта.
- Метод IsValidPhoneNumber: проверяет корректность Номера телефона.
- Метод IsValidName: проверяет корректность Фамилии, имени и отчества.
- Метод IsValidAddress: проверяет корректность Адреса проживания.
- Метод IsValidDate: проверяет корректность Даты рождения и регистрации

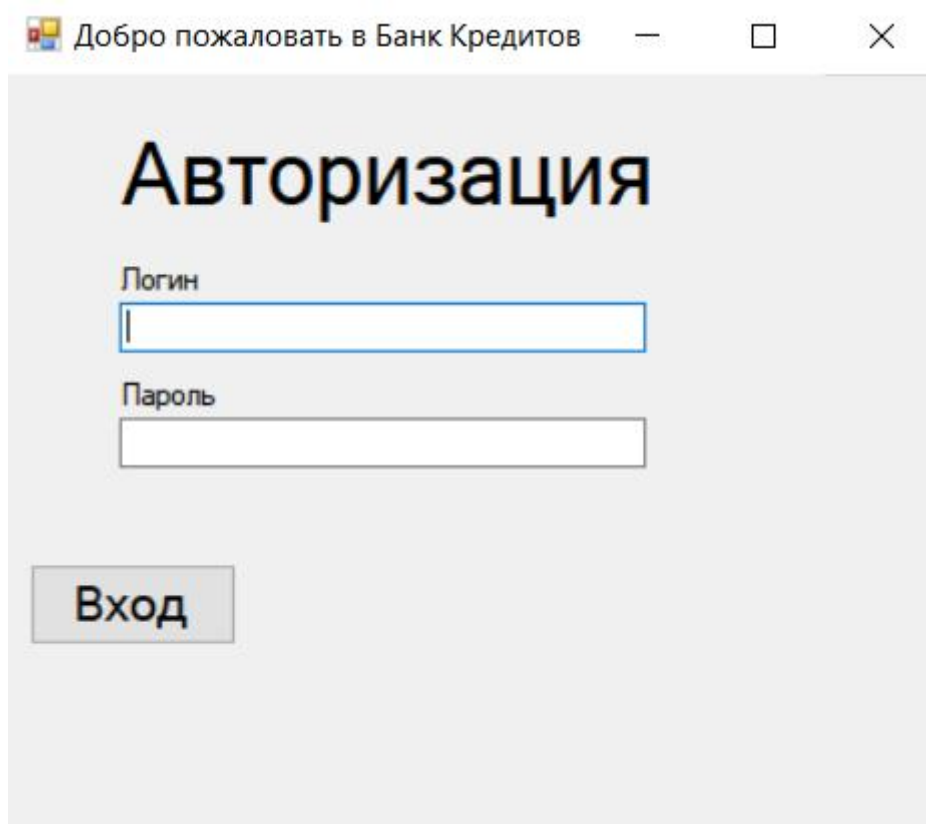
- Метод button17_Click: обрабатывает нажатие кнопки и очищает поле поиска в dataGridView1.
- Метод button18_Click: обрабатывает нажатие кнопки и очищает поле поиска в dataGridView2.
- Метод button19_Click: обрабатывает нажатие кнопки и очищает поле поиска в dataGridView3.
- Метод button20_Click: обрабатывает нажатие кнопки и очищает поле поиска в dataGridView4.

4) Form4.cs

- Конструктор Form4: инициализирует форму, устанавливает соединение с базой данных и заполняет ComboBox 1 необходимыми данными.
- Метод LoadPlatechData: выполняет SQL-запрос и заполняет dataGridView1 данными об платежах.
- Метод button1_Click: обрабатывает нажатие кнопки поиска, выполняет SQL-запрос и выводит результат в dataGridView1.
- Метод InsertPlatechDataToDatabase: выполняет SQL_запрос и добавляет данные в dataGridView4.
- Метод comboBox1_SelectedIndexChanged: выполняет SQL_запрос и заполняет combobox заданными значениями.
- Метод LoadClientsToComboBox: выполняет SQL_запрос и заполняет combobox заданными значениями.
- Метод dataGridView1_SelectionChanged: обрабатывает выбор строки в dataGridView1 и переносит данные в textbox и combobox.
- Метод button2_Click: обрабатывает нажатие кнопки поиска, выполняет SQL-запрос и выводит результат в dataGridView1.
- Метод button3_Click: обрабатывает нажатие кнопки и очищает поле поиска в dataGridView1.
- Метод LoadColumnNames: заполняет combobox наименованиями из dataGridView1.

4. Тестирование АИС

Тестирование автоматизированной информационной системы — ключевой этап разработки, обеспечивающий ее корректность, надежность и безопасность. Первым шагом взаимодействия пользователя с приложением является окно авторизации (рисунок 37), предназначенное для идентификации и аутентификации. Этот элемент играет важную роль в защите данных и управлении доступом к функционалу системы, что делает его приоритетным объектом тестирования.



Добро пожаловать в Банк Кредитов

Авторизация

Логин

Пароль

Вход

Рис. 37 – Окно авторизации

В окне авторизации пользователь вводит логин и пароль своей учетной записи. Если введенные данные корректны, открывается рабочее окно. В случае некорректного логина или пароля приложение выводит сообщение об ошибке с указанием на неверные учетные данные. Иллюстрация этого процесса представлена на рисунке 38.

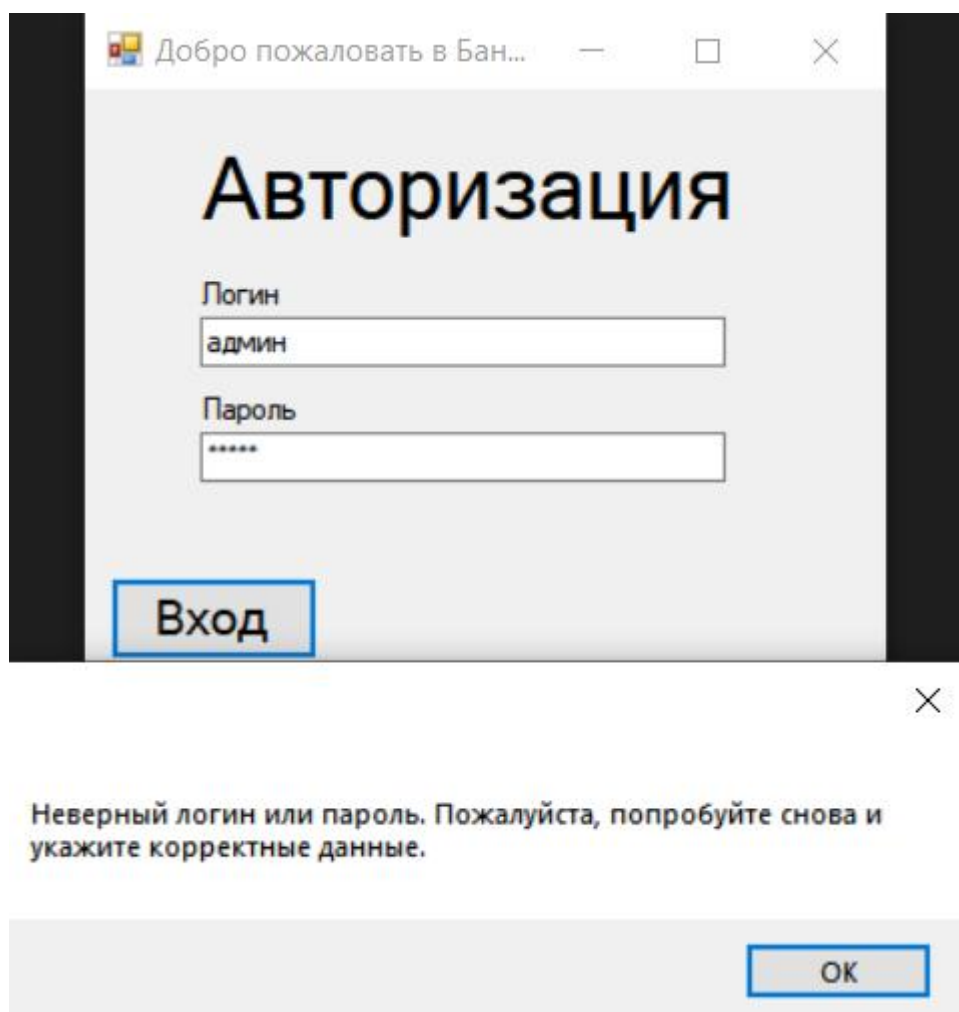


Рис. 38 – Ошибка входа при неправильном логине или пароле

Если логин и пароль правильные, то пользователь попадает в одно из рабочих окон. Начнем с формы клиента. Здесь пользователь может оформить кредит, а после успешного оформления найти его с помощью поиска. Если запись не нашлась, то будет выведено сообщение. Результат представлен на рисунках 39-40.

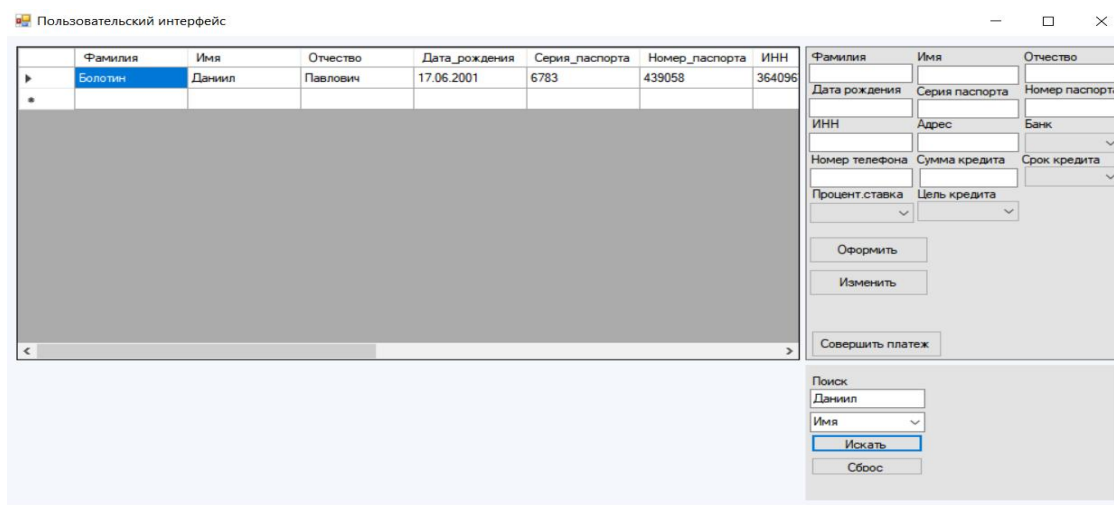


Рис. 39 – Корректный поиск

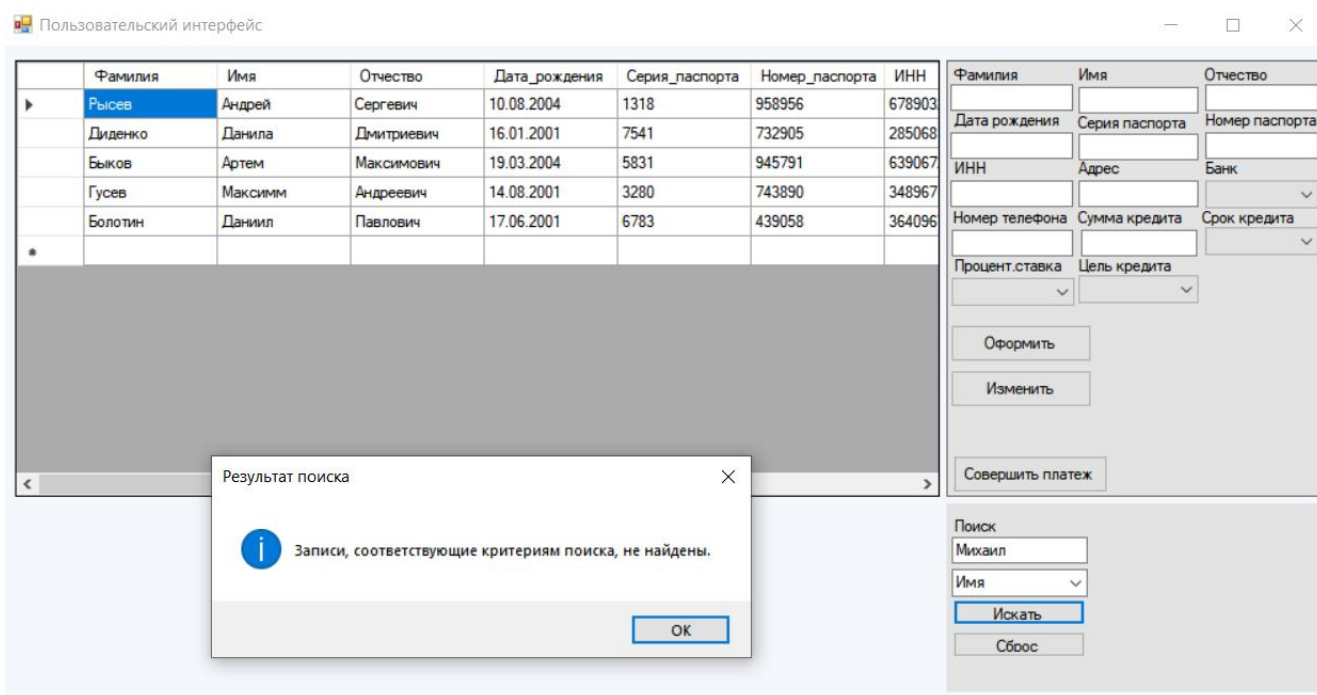


Рис. 40 – Записи не найдены

Далее клиент может выбрать желаемый кредит, указать сумму и процентную ставку, но для этого ему нужно внести свои данные, т.к ФИО, номер телефона и т.п. Если клиент по ошибке ввел некорректные данные, то ему выдаст сообщение о том, что нужно ввести правильно данные. Результат показан на рисунке 41.

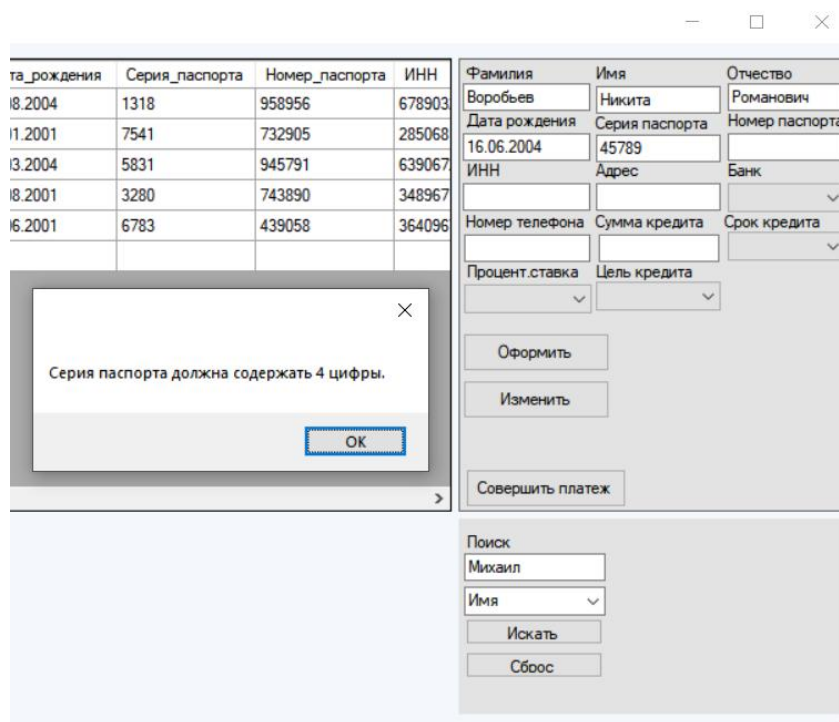


Рис. 41 – Сообщение при неправильном заполнении полей

Если все данные корректны, то при нажатии на кнопку «Оформить» будет выведено сообщение об успешном оформлении кредита, что показано на рисунке 42.

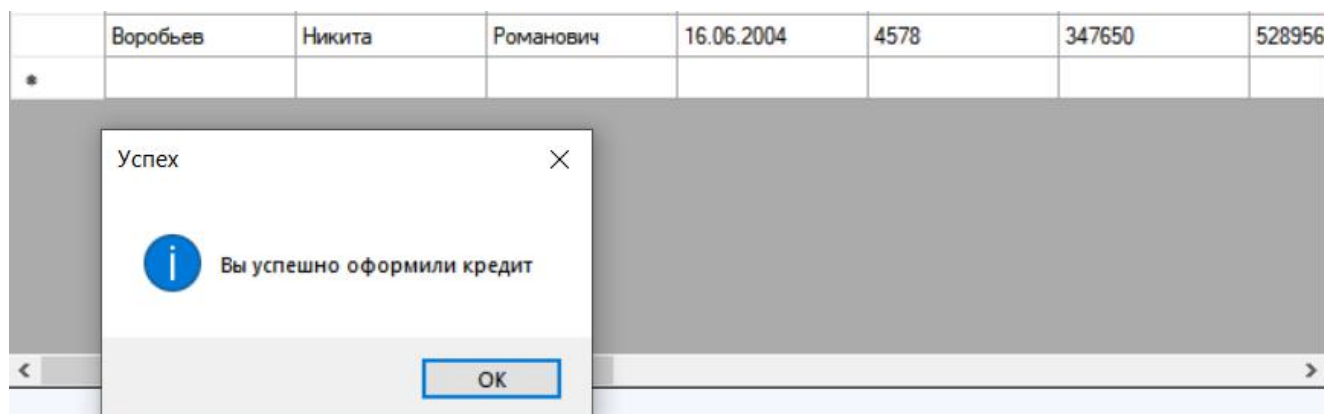


Рис. 42 – Сообщение об успешном оформлении

Теперь перейдем к форме администратора. Он может найти нужные ему данные с помощью поиска. Если запись не нашлась, то будет выведено сообщение. Результат представлен на рисунках 43-44.

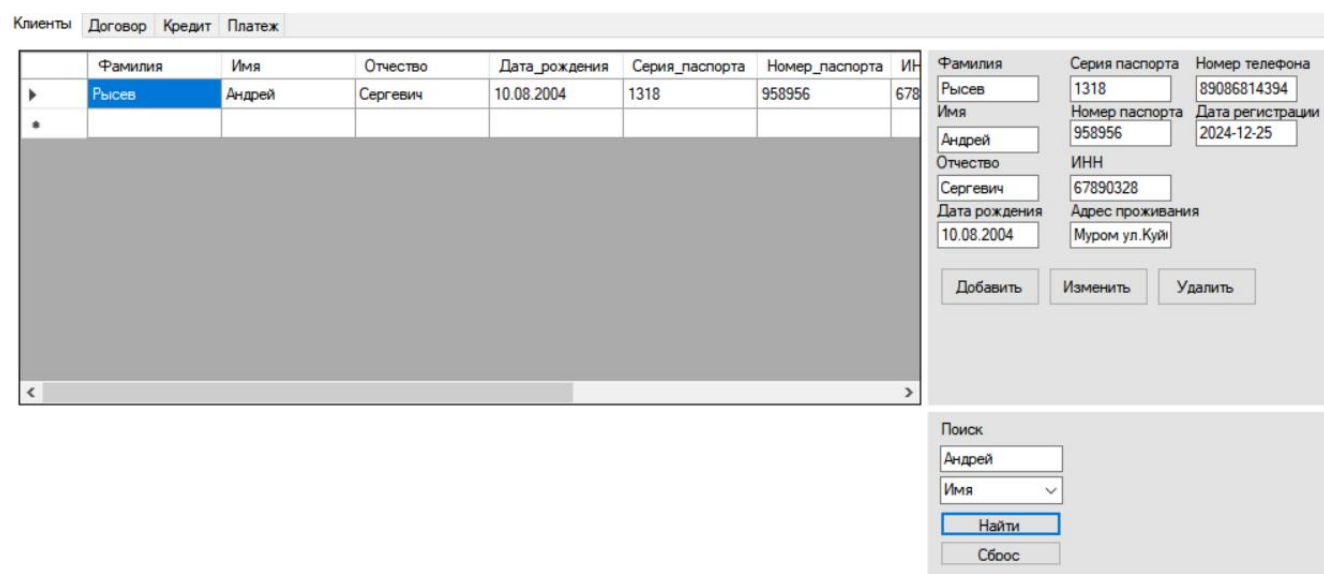


Рис. 43 – Пример корректного поиска

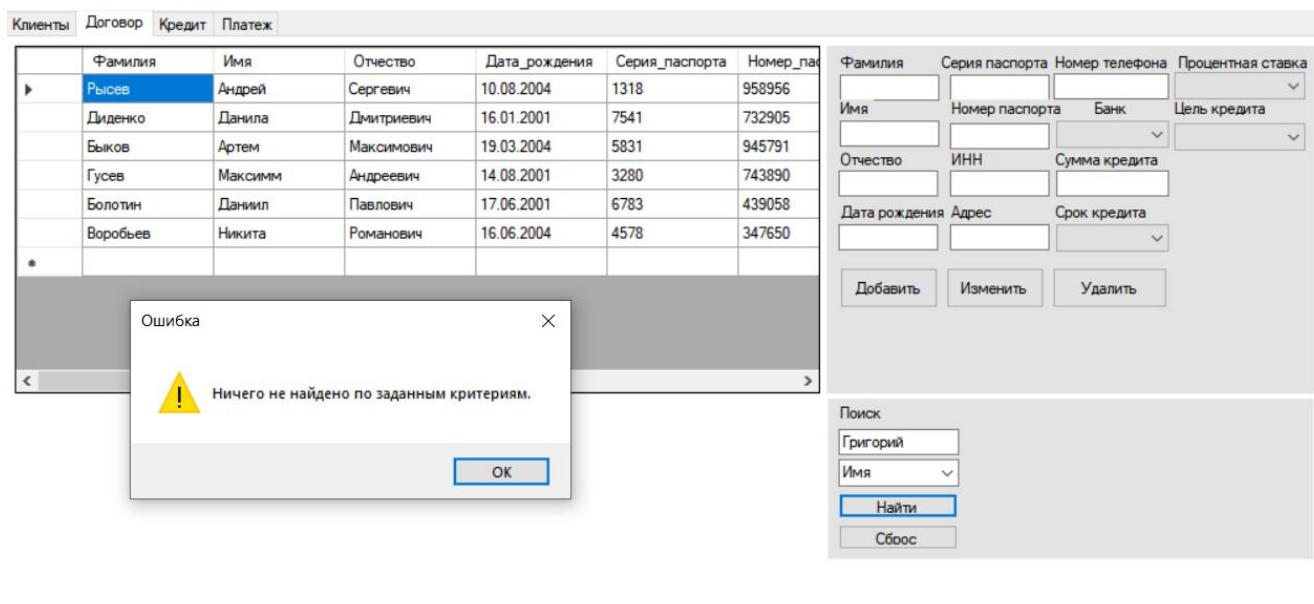


Рис. 44 – Записи не найдены

В данном случае показан пример поиска на таблицах Клиент и Договор. Для всех остальных таблиц он работает аналогично.

Также администратор может редактировать или удалять записи. Для этого ему нужно выбрать запись, данные будут перенесены в текстовые поля. При нажатии на кнопку они будут изменены или удалены. Результат показан на рисунках 45-46.

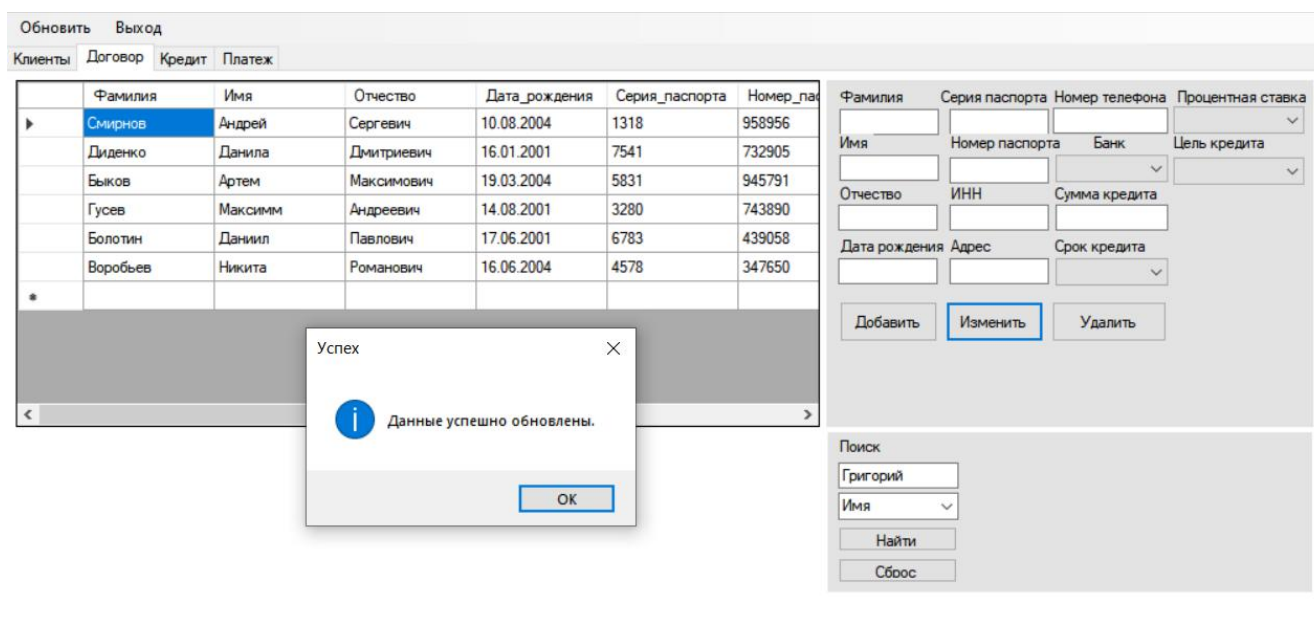


Рис. 45 – Обновление записи

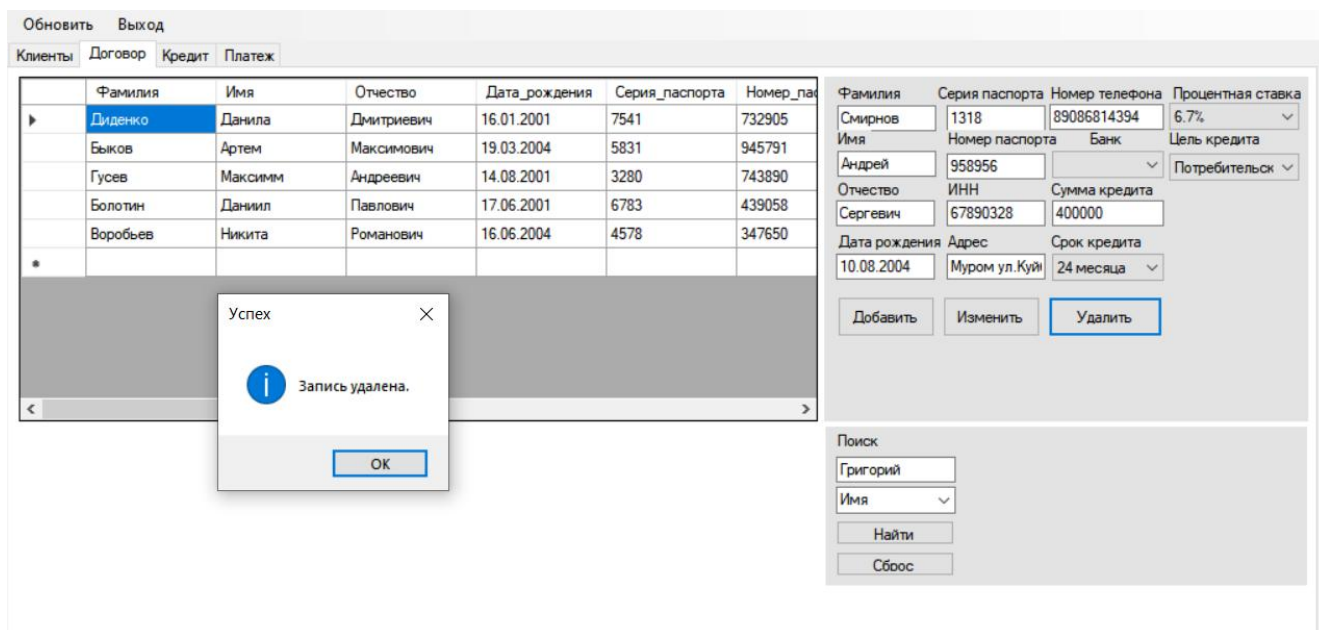


Рис. 46 – Удаление записи

В данном случае показан пример поиска на таблице Договор. Для всех остальных таблиц он работает аналогично.

В меню при нажатии на кнопку «Обновить» администратор обновит все таблицы. При завершении будет выведено сообщение, показанное на рисунке 47.

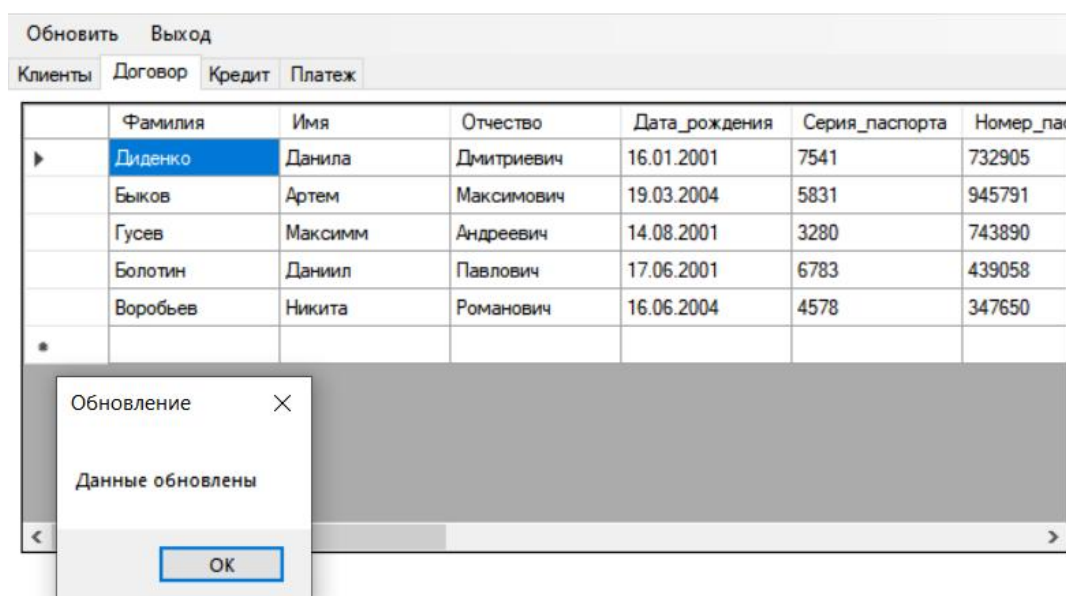


Рис. 47 – Обновление данных

Заключение

Во время данной курсовой работы были решены поставленные задачи, направленные на создание Автоматизированной Информационной Системы (АИС) для отдела кредитования. Основное внимание уделялось разработке базы данных, обеспечивающей эффективное управление информацией. Использование такой системы позволяет автоматизировать процессы кредитования и оформления соответствующих документов.

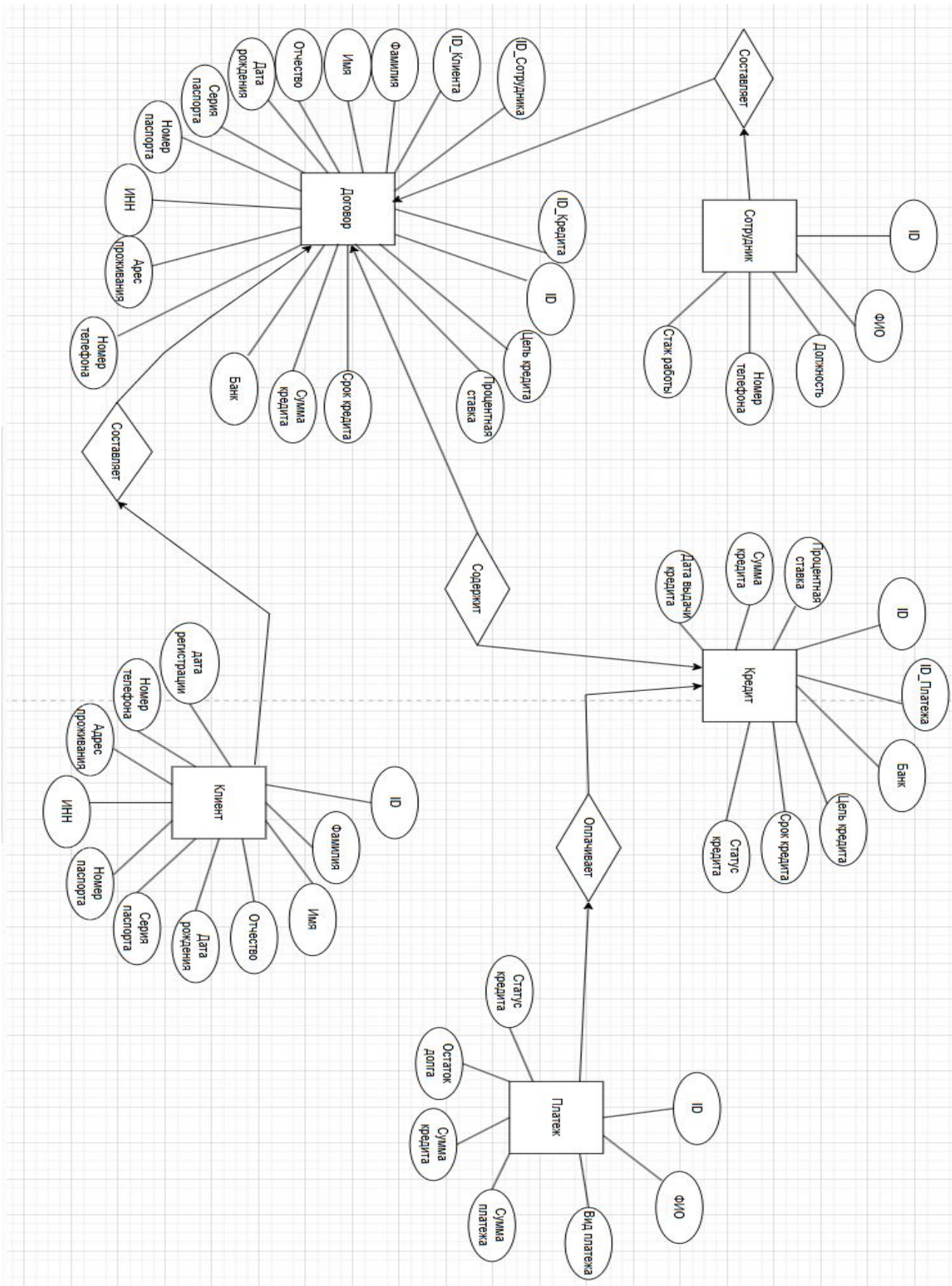
Разработанная АИС также значительно улучшила взаимодействие между клиентами и сотрудниками отдела кредитования. Каждый клиент теперь имеет возможность оперативно получать информацию о доступных кредитных программах и условиях, что повышает уровень удовлетворенности и доверия к услугам отдела кредитования.

На основе проведенного сравнительного анализа производительности различных систем управления базами данных и языков программирования были выбраны оптимальные решения для разработки данной базы данных и многопользовательского приложения отдела кредитования. Это позволило создать надежную, эффективную и масштабируемую систему, готовую к дальнейшему развитию.

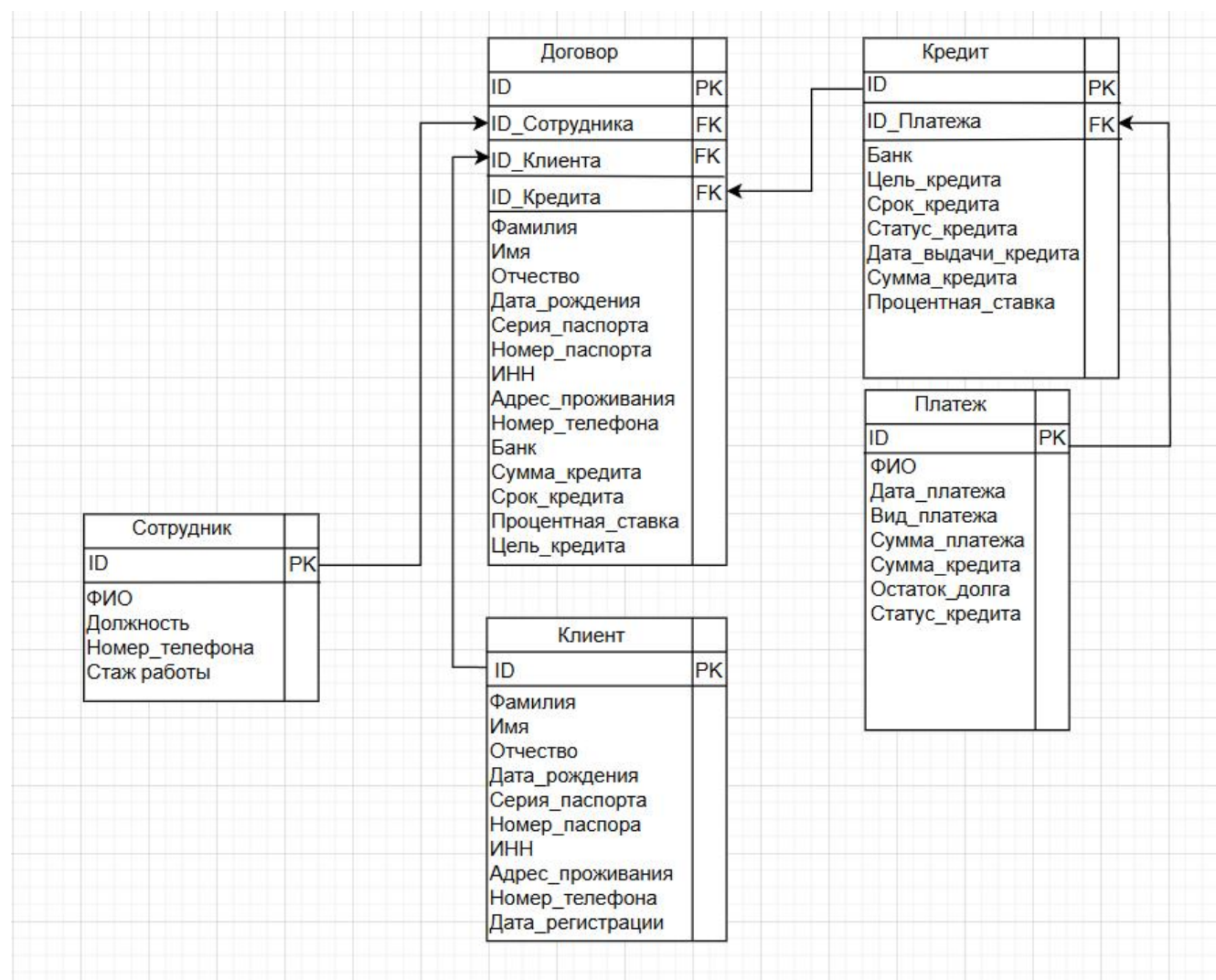
Список литературы

- 1) Бьерн Страуструп, Язык программирования C++. Специальное издание. Пер. с англ. – М.: Издательство Бином, 2011 г. – 1136 с.: ил.
- 2) Медведев, М. А. М42 Программирование на СИ# : учеб. пособие / М. А. Медведев, А. Н. Медведев. — Екатеринбург : Изд-во Урал. ун-та, 2015. — 64 с.
- 3) Цифровые технологии управления получением, хранением, передачей и обработкой больших данных: SQLite : учеб.-метод. пособие / С. В. Рындина. — Пенза : Изд-во ПГУ, 2023. — 82 с.
- 4) Мотев А. А. М85 Уроки MySQL. Самоучитель. — СПб.: БХВ-Петербург, 2006. — 208 с.: ил.
- 5) Разработка Windows Forms приложений на языке программирования C#: учебно-методическое пособие / Сост.: А.Ю. Сапаров, Ижевск, 2020. 61 с.
- 6) Казарин С.А., Клишин А.П. К 143 Среда разработки Java-приложений Eclipse: (ПО для объектно-ориентированного программирования и разработки приложений на языке Java): Учебное пособие. Москва 2008. — 77 с.
- 7) Сергеева Т.И. Базы данных: модели данных, проектирование, язык SQL: учеб. пособие / Т.И. Сергеева, М.Ю. Сергеев. Воронеж: ФГБОУ ВПО «Воронежский государственный технический университет», 2012. 233 с.

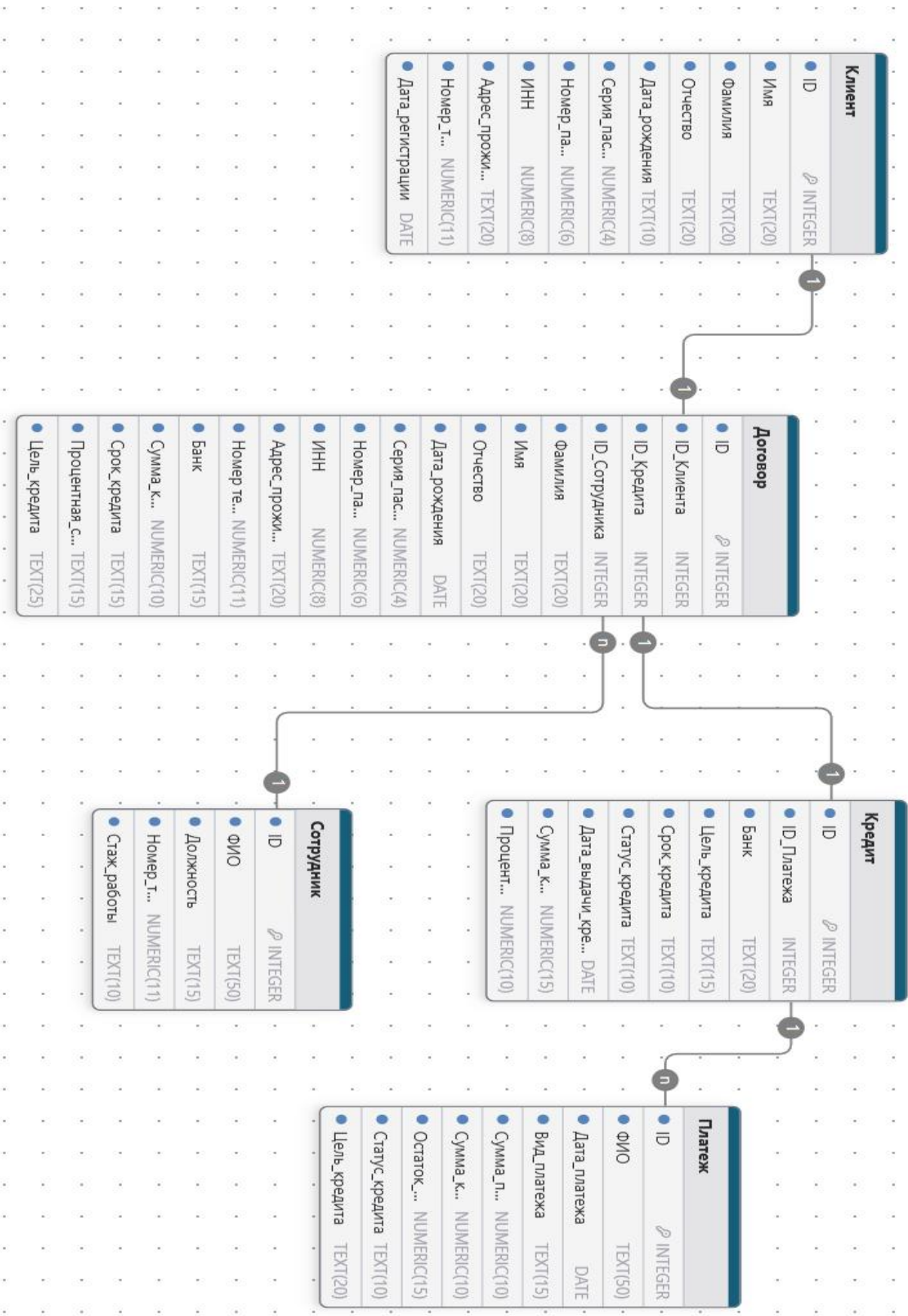
Приложение А: «Модели данных»
Концептуальная модель.



Логическая модель.



Физическая модель.



Приложение Б: Текст кода

Весь исходный код представлен по ссылке:

https://github.com/KyCyMa/BD_Bank_credit

					МИВУ 10.03.01-12ПЗ	Лист
						70
Изм.	Лист	№ докум	Подпись	Дата		